

Manual

&

Theory of operation

Welcome

What is openDAQ?

OpenDAQ is an USB based measurement and automation device, which provides multifunction data-logging capabilities such as analog inputs and outputs, digital inputs and outputs, timers, counters and many more.

OpenDAQ is an open source acquisition instrument, which allows users to customize their application in an unprecedented manner. OpenDAQ is intended to be a low-cost, open, and easy to use interface between your computer and the physical world.

OpenDAQ is available in two different hardware versions: [M] and [S]. The [M] version has superior performance, in terms of bits of resolution, input impedance, and output bipolar voltage swing capability.

Features

- 8 Single-Ended or Differential, 16-Bit Analog Inputs
- Up to ±12 Volt Analog Input Range
- Selectable PGA gains: +1, +2, +10, +100V/V
- Up to 20kSamples/Sec
- Software or Hardware Timed Acquisition
- Instrumentation Amplifier Inputs (high impedance)
- 14-bit Analog Output, ±4V Range
- 6 Digital I/O
- Capture Mode for Frequency-to-Voltage Conversion
- 16-Bit Counter Input
- Pulse Width Modulation -PWM- Output (10bit)
- Encoder Input
- Waveform Generation: Sine, Triangle, Sawtooth, Square Signal Shapes
- Watchdog Timer Function
- Open-Source Firmware, User Editable
- SPI bus (bit-bang) communications
- Ultra Small-Size: approx. 50x50mm (1.97”x1.97”)
- Anodized Aluminum Case With Laser Marking
- USB 2.0/1.1 Low Speed Interface, VCP
- 100% Software Controlled, No Jumpers or Switches
- USB Powered Device (No External Power Supply Needed)
- Power Supply Output (+5V USB voltage output)
- Includes Sample Applications for Python and LabVIEW
- Works with Almost Every 32 and 64 bit O.S.
- Includes Cable and Screwdriver
- Money Back Guarantee

Applications

Read and display sensors that output voltage or current, like: temperature, pressure, acceleration, humidity, gas concentration, light intensity sensors, and many more.

Read and display sensors that output digital signals, like: linear and rotary encoders, proximity, flow, vibration, contact sensors, and many more.

Control and interact with devices such as motors, lights, relays or valves.

Generate waveforms as excitation signals for testing electronic devices, or even for artistic purposes.

OpenDAQ brings all the information that it captures to a host computer, by means of an USB connection, where the user can decide how to process, display and store it.

OpenDAQ users also have access to the internal firmware of the device, which allows them to customize device functionalities in an unprecedented manner.

Box Contents

- 1 openDAQ device
- 1 USB cable (80 cm length)
- 1 Screwdriver



Hardware

Having a look from outside, we will find the following different parts in openDAQ:

1. USB connector: it is a mini-B type connection for USB cable. All power and communication of the device are handled by the USB terminal.
2. Right side screw terminal block: it has connections for 4 analog inputs (A1 to A4), 4 digital input/outputs (D1 to D4), apart from 2 ground connections, analog and digital grounds respectively.
3. Left side screw terminal block: another 4 analog inputs (A4 to A8), 2 more digital connections (D5 and D6), 2 ground connections, DAC output connection and a +5V output supply connection.
4. Status LED: on top of the box, there is a two-color LED indicator, which will be normally shining green when the device is connected. A red color will indicate some type of problem (overload/overvoltage). It also will flick in orange (green & red) when collecting data in stream mode.

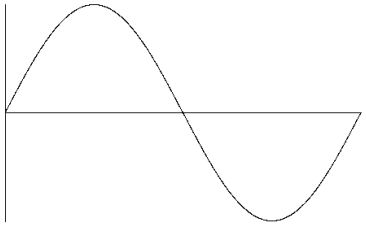


Analog Inputs

An analog input converts a voltage level into a digital value that can be stored and processed by the software.

Analog acquisition is the main feature for openDAQ. The most typical application for these inputs is reading data from external sensors. There are quite a lot of sensors available which convert their readings, like temperature, pressure, etc. into a voltage output.

In other cases, the sensor output is not directly a voltage, but a current or a change in impedance. In those cases, you will need one or two external components and connections to convert that magnitude into a voltage that can be read through the analog inputs.



Features

- 8 analog inputs (multiplexed)
- Analog input range is ±12V
- Differential of single-ended channels
- Analog input resolution is 16 bits (14 bits in case of model [S])
- Up to 20k samples per second
- Instrumentation amplifier input
- Software selectable gains of up to x100 amplification (x20 for model [S])
- Input impedance is at least 1 GΩ (model [M])

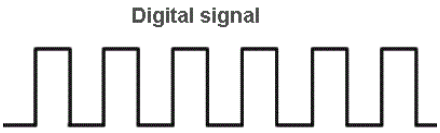
Digital I/O

A digital input provides a voltage thresholding operation. If the voltage is higher than a certain value, the device will detect the digital input as “high” or logic “1”. If the voltage is lower than another specific value, the device will detect the digital input as “low” or logic “0”. In the case of openDAQ, it will detect a Low Input if the signal is below 1.5V and a High Input if the signal is above 3V approximately.

Digital inputs may be used to detect changes in the state of an external sensor or actuator (for example, a button press).

A digital output allows you to control a voltage with a computer. If the computer instructs the output to be high, the output will produce a voltage (5Volts). If the computer instructs the output to be low, it is connected to ground and produces no voltage. OpenDAQ outputs will always be above 4.5V for High Outputs and below 0.5V for Low Outputs.

Digital outputs are useful to control the state of external actuators like, for example, a relay or a light.



Features

- 6 configurable digital input-output connections (D1-D6)
- 5V TTL logic compatible
- Overvoltage/short-circuit protected
- 100Ω series equivalent resistance

Analog Output

An analog output converts digital values from the computer into a voltage level. This provides an adjustable output value.

It can be used to generate specific reference patterns such a sine or triangular wave. This output is a very flexible mean to interact with external sensors and components.

Features

- 14 bit DAC generator (12 bit for model [S])
- ±4.096V signal range (0 to 4.096V in case of model [S])
- 100Ω series equivalent resistance

Capture Input

The capture input permits measuring the time length of incoming digital signals.

It makes use of device internal timer to calculate the time elapsed between changes in state (high to low or low to high) of an external signal. OpenDAQ has a main clock running at 16MHz, which limits the minimum periods that the device is able to measure to several microseconds.

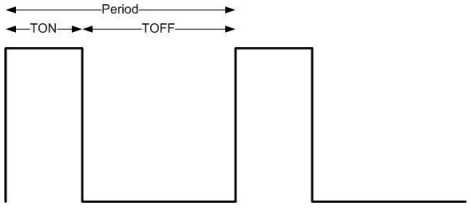
Features

- Internally connected to DIO 5 (D5)
- 16 bit internal timer (16MHz)
- 3 modes: high time, low time, total period
- Microseconds resolution

PWM Output

Pulse Width Modulator generates a continuous digital signal at a given frequency. Duty refers to the portion of time that the signal spends in High state.

PWM can be used as an alternative way to generate an analog output. Controlling the duty and adding an external low-pass filter that rectifies the digital pulses, an analog signal going from 0 to 5V can be generated. PWM is also commonly used to control stepping motors. Output impedance is about 1kΩ.



Period = TON + TOFF

Frequency = 1 / Period

Duty Cycle = $\frac{TON}{TON + TOFF}$ * 100

Features

- Duty value passed should be 0-1024, and determines what portion of the total time is spent high.
- Shares Timer with capture input and counter input.
- Internally connected to D5.

Counter Input

This input just counts and stores the number of state changes in a external digital signal, from low to high, or from high to low.

It can be very useful to read certain kinds of sensors, like encoders, to get information about position, counting the total amount of pulses, or even the velocity, counting number of pulses for a given amount of time.

Features

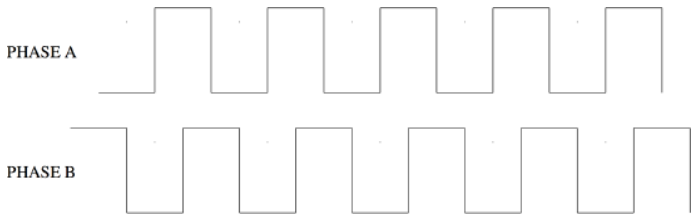
- 16bit internal accumulator.
- Shares Timer with capture input and PWM output.
- Internally connected to D5.

Encoder Input

OpenDAQ has a built in feature that allows straight reading of digital quadrature encoders.

Encoders are position and orientation sensors, commonly used as a reference or active feedback to control position. They are commonly based on magnetic or optic means. There are linear and also rotary encoders and they may be absolute or incremental.

The most common type of incremental encoder uses two output channels or phases (A and B) to sense position. Using two code tracks with sectors positioned 90 degrees out of phase, the two output channels of the quadrature encoder indicate both position and direction of rotation. By monitoring both the number of pulses and the relative phase of signals A and B, the position and direction of rotation can be tracked.



OpenDAQ makes use of external interrupts of its microcontroller to read this type of signal. Quadrature encoders must be connected to digital pins D5 and D6. D6 keeps track of the number of pulses given by the encoder (position) and D5 is used to check the displacement between signals (direction).

Features

- 16bit internal accumulator.
- Uses external interrupts.
- Internally connected to D5 and D6.

GND and AGND

The GND connections available at the screw-terminals of openDAQ is the same as the ground line on the USB connection, which is often the same as ground on the PC chassis and therefore AC mains ground. AGND shares 0V reference with GND, and they are connected in series through a coil, which limits the high frequency (MHz) noises going between analog and digital parts of the circuit.

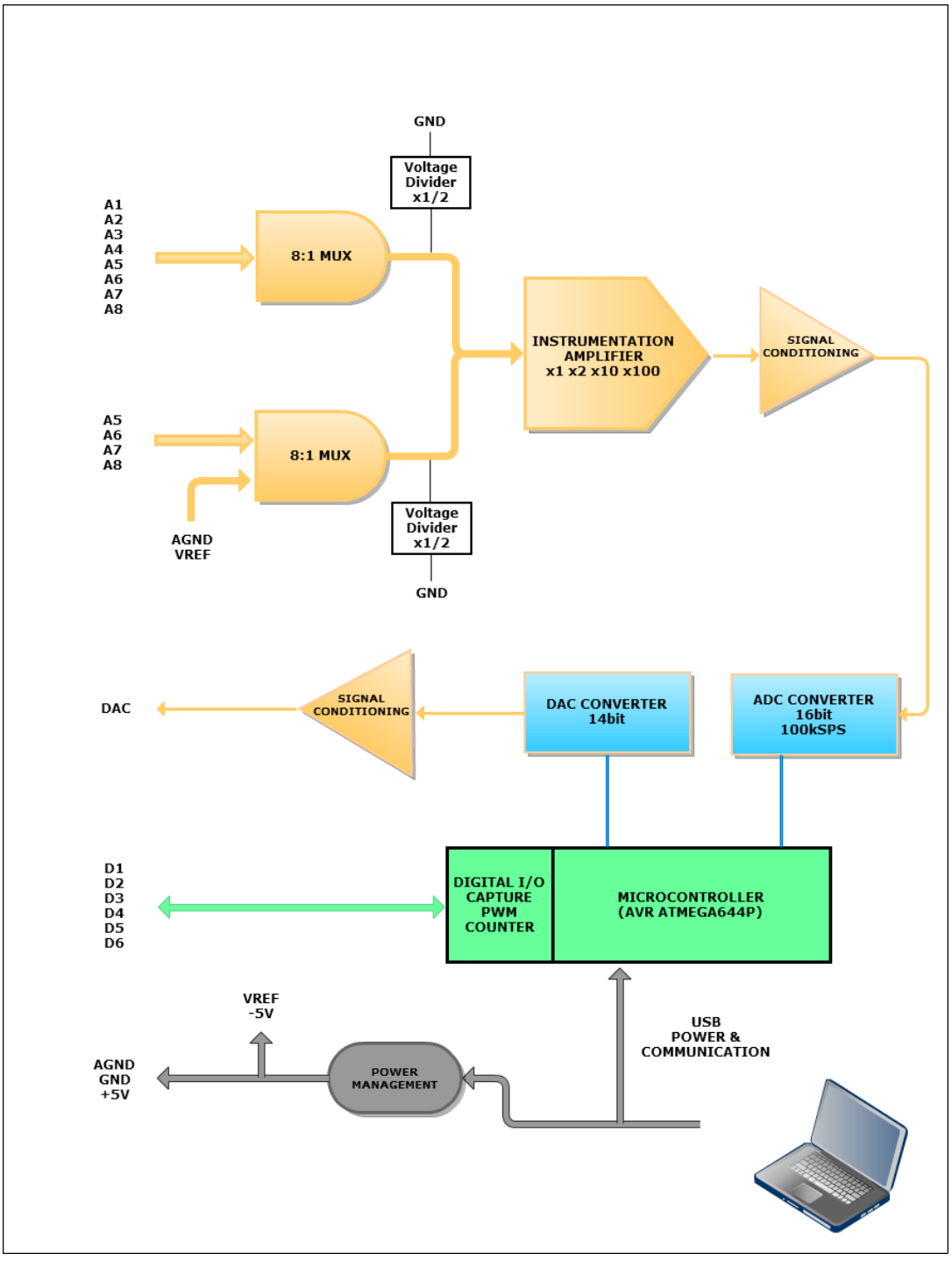
Status LED

The two-color LED indicator on top of the device will be normally shining green when the device is connected. In Command-response Mode, user can manually control the operation of this LED and change its color at any time.

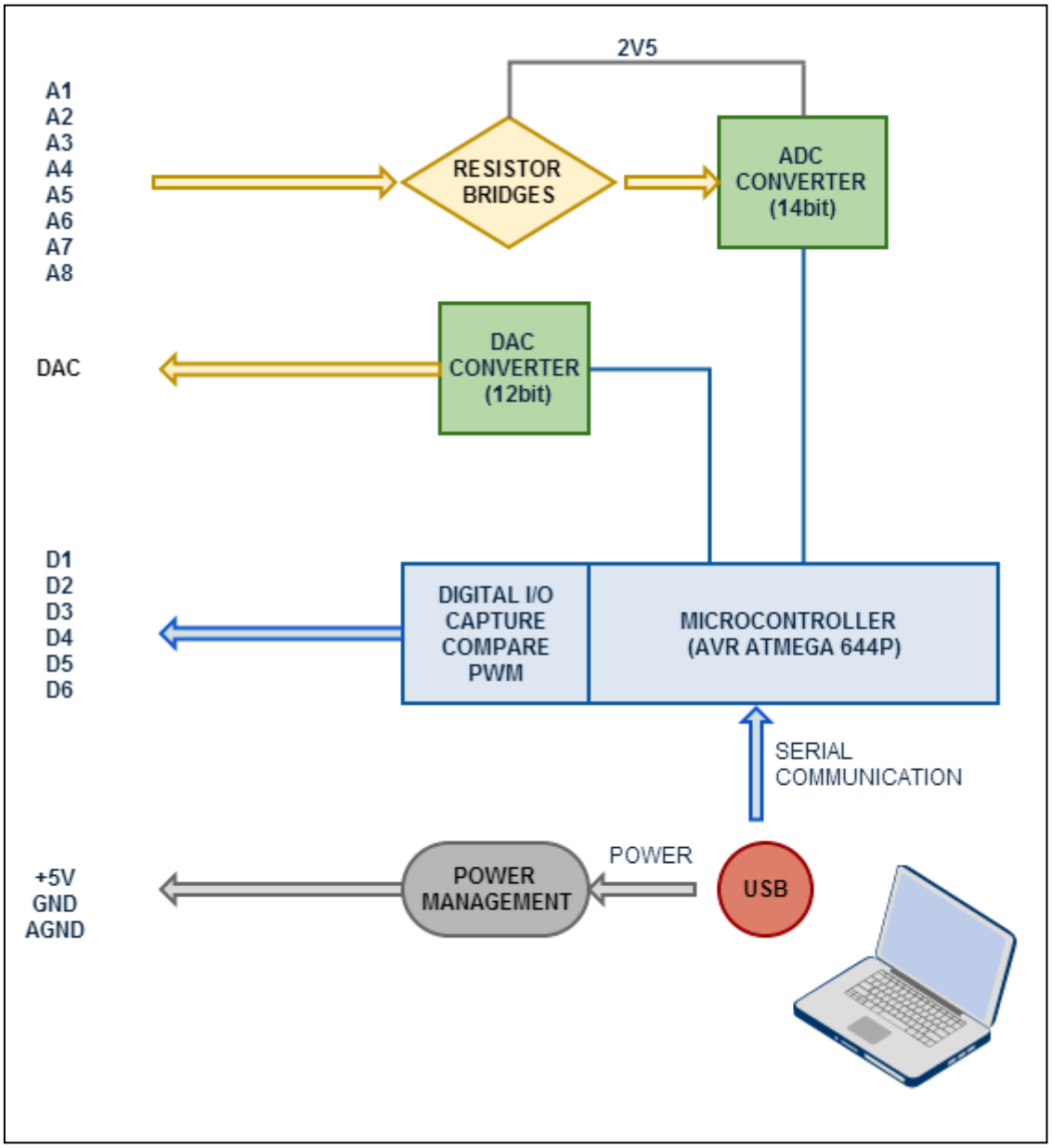
In Stream Mode, the LED will flick in orange (green + red) when collecting the data points. A red color will indicate some type of problem in the measurements (overload/overvoltage).

Block Diagram

A general functional block diagram of openDAQ [M] is shown in the following figure:



In the following figure it is shown the same diagram for the case of openDAQ [S]:



Electrical Data – Model [M]

Analog Inputs	
Channels	8
Single-Ended Channels	8
Differential Channels	4
Resolution	16 bits
Max. Sample Rate	100 kS/s
Throughput (All Channels)	7.2 kS/s
Maximum Voltage	+-12V
Voltage Ranges	±12V, ±4V, ±2V, ±0.4V, and ±0.04V
Maximum Voltage Accuracy	2 mV
Simultaneous Sampling	No
On-Board Memory	1600 points

Analog Output	
Channels	1
Resolution	14 bits
Max Voltage	4 V
Maximum Voltage Range	±4V
Maximum Voltage Range Accuracy	3 mV
Update Rate	100 kS/s
Current Drive Capacity	10 mA

Digital I/O	
I/O Channels	6
Logic Levels	TTL
Maximum Voltage Range	0 V - 5 V
VIH	3.5V
VIL	1V
VOH	4.5V
VOL	0.5V
Current Drive Single	10mA
Current Drive All	60 mA

Counter/Timer	
Counters	1
Maximum Range	0 V - 5 V
Max Source Frequency	1 MHz
Resolution	16 bits
Pulse Generation	Yes, 10 bits
Timebase Stability	30 ppm
Logic Levels	TTL

Mechanical Specifications	
Length	5 cm
Width	5 cm
Height	1 cm
Material	Anodized aluminium
I/O Connector	Screw terminals

Timing/Triggering	
Triggering	Digital, Analog
Watchdog Timer	Yes

Electrical Data – Model [S]

Analog Inputs	
Channels	8
Single-Ended Channels	8
Differential Channels	4
Resolution	14 bits
Max. Sample Rate	48 kS/s
Maximum Voltage	±12V
PGA ranges (DE mode)	1, 2, 4, 5, 8, 10, 16 and 20V/V
Internal reference	2.5V
Maximum Voltage Accuracy	5 mV
Simultaneous Sampling	No
On-Board Memory	1600 points

Analog Output	
Channels	1
Resolution	12 bits
Maximum Voltage Range	0 to 4.096V
Maximum Voltage Range Accuracy	3 mV
Update Rate	100 kS/s
Current Drive Capacity	10 mA

Digital I/O	
I/O Channels	6
Logic Levels	TTL
Maximum Voltage Range	0 V - 5 V
VIH	3.5V
VIL	1V
VOH	4.5V
VOL	0.5V
Current Drive Single	10mA
Current Drive All	60 mA

Counter/Timer	
Counters	1
Maximum Range	0 V - 5 V
Max Source Frequency	1 MHz
Resolution	16 bits
Pulse Generation	Yes, 10 bits
Timebase Stability	30 ppm
Logic Levels	TTL

Mechanical Specifications	
Length	5 cm
Width	5 cm
Height	1 cm
Material	Anodized aluminium
I/O Connector	Screw terminals

Timing/Triggering	
Triggering	Digital, Analog
Watchdog Timer	Yes

Firmware Architecture

Main concepts

The openDAQ's architecture is based on the Atmel ATMEGA 644P microcontroller. To allow users to easily build their own routines and programs, openDAQ firmware core is similar and compatible to a certain extent with Arduino's libraries and source code.

Arduino is an open source microcontroller board, descendant of the also open-source Wiring platform. It was designed to make the process of using electronics in multidisciplinary projects more accessible. It is programmed using a Wiring-based language (syntax and libraries), similar to C++ with some slight simplifications and modifications, and a Processing-based integrated development environment (IDE). It is based on the standard GCC compiler for AVR microcontrollers (AVR-GCC).

Arduino boards also include a serial bootloader that simplifies the process of loading of the programs into the microcontroller. When the board is connected to the PC, it starts the bootloader, and there is a waiting time (62.5ms) before beginning the first instruction, that is used to start loading new compiled programs updates. If a new program is received from the USB during this time, it will be loaded into the FLASH memory (64KB) substituting already existing programs. Otherwise, if a new program is not received, the last program stored in the memory will start running.

OpenDAQ core is based on AVR ATMEGA 644P microcontroller. Low level libraries and files are based on a non-official Arduino branch called Sanguino. Internal libraries and hardware definitions have been modified to the characteristics of openDAQ circuit. Instructions to modify the Arduino IDE, and the specific libraries and definitions needed for openDAQ can be downloaded from our repositories.

OpenDAQ device also includes a bootloader, so new firmware can be loaded to the device directly through the USB connection. The USB uses a VCP (virtual COM port) driver that emulates a serial connection.

Operating Modes

OpenDAQ has two main modes of operation: Command-Response mode, and Stream or hardware-timed mode.

Command-response mode is where communication is initiated by a command from the host PC, which is followed by a response from openDAQ. Command/response is generally slower and simpler than stream mode.

Stream mode is a continuous hardware-timed input mode where a list of channels is scanned at a specified scan rate. The scan rate specifies the interval between the beginning of each scan, and it can be defined by an internal hardware timer or by an external trigger connected to one of the digital inputs.

Command/Response mode is generally best for minimum-latency applications such as feedback control. By latency here we mean the time from when a reading is acquired to when it is available in the host software. Stream mode is generally best for maximum-throughput applications where latency is not so important.

Command/Response can be done while streaming, but streaming needs exclusive control of the analog input system, so analog inputs cannot be read via command/response while a stream is running.

Stream mode can refer to three different kinds of experiments, which differ in the maximum scan rate allowed and the source of the timing clock (internal or external). We define an experiment as a certain data source with specific configuration, with a specific sampling rate, and start and stop conditions:

- Stream experiments
- External experiments
- Burst experiments

STREAM Experiments

For Stream Experiments, a specific data source is sampled in regular intervals, using internal timer to keep time count (Timer 2). Fastest scan rate in this mode is 1kSPS (1ms of period).

User can configure up to 4 Stream experiments to be running simultaneously. They will have each an internal buffer of about 400 samples, which will be normally enough not to lose any point in the communications.

Features

- 1ms to 60s interval time
- Event types:
    - Analog input
    - Analog signal output
    - Digital input or output
    - Counter input
    - Capture input
- Up to 4 experiments simultaneously (including EXTERNAL experiments)

EXTERNAL Experiments

External experiments use an external digital trigger source to perform readings. Fastest scan rates, however, are intended to be in similar ranges as for the Stream experiments. The rest of properties and parameters are like Stream experiments.

User can define up to 4 external experiments at the same time, each of one connected to digital inputs D1 to D4 (the number of the experiment is connected to the digital input number) to act as trigger inputs. Maximum number of experiments will be 4 in total, including all External and Stream experiments.

Features

- External timed: up to four experiments connected with digital inputs D1 to D4
- Configurable for rising or falling edges
- Event types:
    - Analog input
    - Analog signal output
    - Digital input or output
    - Counter input
    - Capture input
- Up to 4 experiments simultaneously (including STREAM experiments)

BURST Experiments

Burst experiments are also internally timed, like External experiments, but they are intended to use a faster sampling rate, up to 20kSPS. The high acquisition rate limits the amount of things that the processor is capable of doing at the same time. Thus, when a Burst experiment is carried out, no more experiments can run at the same time.

Burst experiment use a bigger internal buffer of about 1600 points to temporary store results. However, if the experiment goes on for a long time, the buffer will eventually get full and the firmware will enter "Auto-recovery" mode. This means that it will get no more points until buffer gets empty again, having an time where no sample will be taken.

Features

- Internal timed (Timer 2)
- Interval time: from 50µs
- Event types:
    - Analog input
    - Analog output
    - Digital input or output
- Only 1 channel at a time. No simultaneous experiments allowed (external or stream)

DataChannels

DataChannels are the internal structures that actually handle all the information about the experiments that the user wants to perform with the device using stream mode.

They keep information about the data source for the experiment (analog input or output, timer, digital I/O) and its particular configuration. It also has the information about Start and Stop conditions (number of points, repetitions, and trigger mode), scan rate and source.

DataChannels also include the memory buffer for 400 points of input/output data (1600 points in case of Burst experiments), and handle all the low level functions that must be called depending on the experiment configuration.

Source types

- Analog input (AIN+, AIN-, GAIN)
- Analog signal output (signal preload)
- Digital input or output (D1 to D6)
- Counter input (reset counter when reading?)
- Capture input (high/low/combined period, time range)

Datachannel internal states

- IDLE: Waiting for Start command coming from PC.
- READY: Start command was received, waiting for start condition (trigger) to be true.
- RUN: Performing experiment.
- STOP: Experiment finished, waiting for all the data to be transferred and for other simultaneous experiments to end.

Number of points

- Fixed number (1 to 65536)
- Unlimited

Run mode

- Continuous: Start again after reaching maximum number of points.
- Run once: Run until reaching maximum number of points.

Trigger Mode

There are two possibilities to triggering a experiment: software or hardware triggered. Software triggered experiments perform RUN directly after receiving Start command from the host PC. Hardware triggered experiments wait for a specific hardware event, after receiving Start command, to initiate measurements:

- Digital input triggered
- Digital channel (D1 to D4)
- Condition: Edge (falling or rising type)
- Analog input triggered
- Uses actual channel configuration (AIN+, AIN-, GAIN)
- Condition: Signal bigger/lower than defined threshold

Serial Protocol

For reasons of compatibility with Arduino platform, USB handler uses a VCP (Virtual COM Port) configuration. Thus, when the device is connected to a computer, a new COM port is created, and it can be accessed as any other serial port. In order to communicate with openDAQ, the following settings must be configured in the terminal program that you are using:

- Rate: 115200 Baud
- Data Bits: 8
- Parity: None
- Stop Bits: 1
- Flow Control: None

Following is a description of openDAQ's low level protocol. Device communicates with host computer by means of a binary serial protocol. There are two kinds of commands used by openDAQ: Regular Command Format, used for most of the commands, and Stream Data Format, used to transmit large packets of data when the device is running in Stream mode.

Regular Command Format

Normal openDAQ commands or Regular Commands use a minimum of 4 bytes to transmit a message to the device. Maximum allowed length for these packets is 64 bytes.

First two bytes consist on a 16-bit checksum or CRC. This checksum is calculated doing the sum of all other bytes in the packet into a 16-bit integer. Then, the result is complemented.

Be aware that some commands transmit 16 bit integers, and they do it in Big-Endian format (higher bytes first), while Arduino stores variables in its memory in Little-Endian format (lower bytes first).

byte	Description	bits	
0	CRC16H		
1	CRC16L		Sum of all bytes 2-63 complemented with xFFFF
2	Command number	1:255	
3	Number of data bytes	0:60	Total amount of data bytes (4-63)
4:63	Data		NOTE: 16bit words represented in BIG-ENDIAN format, H-L (Arduino uses LITTLE-ENDIAN)

Device will react to all these commands with a response with the same command number and different fields of data depending on the command. (Command-Response mode)

Stream Data Format

Stream Data packets differ from normal commands because the device sends them without being asked to do it, and no further response is expected.

When openDAQ is performing one or more simultaneous experiments, recorded data should be transmitted as fast as possible, in order not to saturate limited internal memory buffers. The device can't wait for the computer to ask for data, but instead it will send that data in packets as fast as possible.

In order to keep synchronization between openDAQ and the host computer, this packets use a special feature called "**escape characters**". An escape character is a byte of data which invokes an alternative interpretation on subsequent characters in a sequence. It is a common resource, typical for some other serial protocols, like PPP (the protocol used to control phone modems).

In this case, we are using hexadecimal number x7E (decimal 126) to indicate the start of a new packet. To ensure that no other byte in the packet has the same value, we use another escape character, x7D (decimal 125). When this character appears, it indicates that following byte must be modified in order to be processed adequately. In this case, the character following a x7D must be exor'ed with x20.

For example, x7E data byte will be transmitted in two bytes: x7D x5E.

byte	Description	value	
0	Init Byte	0x7E **	
1	0		Not used
2	0		Not used
3	Command number	25	
4	Number of data bytes	6:60	
5	Number	d01:d04	Number of the channel used
6	P-channel	d01:d08	Positive/SE channel input (default 5)
7	N-channel	d00,d25, d05:d08	Negative channel input (default 0)
8	Gain	d00:d04	00:x1/2, 01:x1, 02:x2, 03:x10, 04:x100, (default 1)
9-63	Data		Data points (16bit each)

* All bytes with value 0x7E or 0x7D must be escaped (byte^0x20)

** Init byte, so it is not escaped

Analog Inputs

The openDAQ has 8 user accessible analog inputs built-in, four of them in each screw terminal block side. The analog inputs have a constant resolution of 16bit.

Input architecture – openDAQ [M]

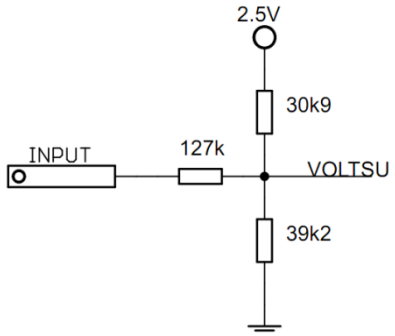
The analog inputs are multiplexed and connected to a high-impedance instrumentation amplifier. For ±12V input range, a voltage divider is used (input signals are divided by 3), and in that case the input impedance is reduced to about 50kΩ. The instrumentation amplifier buffers the signal, allows for single-ended or differential conversions, and provides gains of x1, x2, x10, and x100 (corresponding to ranges of ±4V, ±2V, ±0.4V, and ±0.04V).

Differential readings must use one of the highest 4 inputs for the negative signal (A5 to A8). Positive signal can be any of the 8 analog inputs (A1 to A8).

The inputs are not artificially pulled to ground (except when using ±12V range), as that would reduce the input impedance, so readings obtained from floating channels will generally not be 0 volts. The readings from floating channels depend on adjacent channels and parasitic capacitances, and have little meaning.

Input architecture – openDAQ [S]

Each analog input is connected to a resistor bridge with the values that are seen in the following figure:



This electronic construction coverts the ±12V input range signals into a much smaller signal ranging from 0 to 2.5V, which is directly taken to the ADC converter.

The converter used in openDAQ [S] is the ADS7871, from Texas Instruments. This device has 8 selectable inputs, and integrates the PGA capabilities. In openDAQ legacy firmware, the ADC is configured to use its internal reference of 2.5V.

The ADC converter also offers the possibility to make Single-ended (SE) or differential (DE) readings. The last ones are always configured between a pair of channels: A1-A2, A3-A4, A5-A6 or A7-A8.

Note that a 0V input will result in a different ADC value (VOLTSU=1.231V) than an open terminal (VOLTSU=1.398V).

Note also that PGA amplification has little sense for SE values, as the PGA value would be applied to the VOLTSU signal, not to the INPUT signal, taking 0V as the reference value.

Understanding analog readings

The readings returned by the analog inputs are raw binary values, 16-bit signed integers, for all models. An approximate voltage conversion can be performed as:

VoltsU= (RAW/32768) * Vref

Where the value for Vref is 4.096V in case of openDAQ [M] and 12V in case of openDAQ [S]. That value is the maximum theoretical voltage value that the device is able to read. Note that the model [M] uses a 1/3 voltage divider for reading large values, but the actual range of the inputs is that 4.096V.

The result of that operation is an uncalibrated value, a good approximation, but that will not take into account component imperfections. For a proper voltage conversion, calibration values (Slope and Offset) must be used. These values are stored in the internal flash of the device.

The devices always use the raw values for the internal calculations and data transmission, and it is the external software who has the duty to translate those binary codes into actual voltage values. Those calculations are carried in a slightly different manner depending on the openDAQ model. The code of the conversions is in the *model.py* file in case of the Python library.

Several facts have to be taken into consideration:

- Each analog input will have a different calibration line.
- In the case of openDAQ [M] each gain setting must be calibrated separately, as the gains are set by resistor values with a relatively high tolerance. This is not the case of the openDAQ [S], which uses a PGA with factory calibration for all ranges.
- The inputs of the openDAQ [S] have a different calibration if they are used as single ended (SE) or differential (DE). In the case of openDAQ [M] the calibration can be the same for both modes, because the inputs are just multiplexed.

All of this translates into the following:

- openDAQ [M] has a total of 13 ADC calibration slots, 8 for each analog input, and 5 for each gain setting.
- openDAQ [S] has 16 ADC calibration slots, 8 for each analog input in SE mode, and 8 for each input in DE mode.

See the specific documentation in the "opendaq-python" library or other software packages, to see how these coefficients of calibration are calculated and applied.

Analog connections

Here it is a guide of some parameters that must be considered when trying to read things with openDAQ:

- Voltage: The analog inputs measure the voltage difference between two channels, or the difference between that input and ground (0 volts). For differential readings, both inputs must be within the maximum voltage range for openDAQ, if it is selected, or within ±4V if any other range is selected. Besides, the voltage difference between both inputs must be according the selected range. For single ended readings, the input must be according the range. When measuring parameters other than voltage, or voltages too big or too small for openDAQ, external circuit must be used (i.e. a resistor bridge). You will find examples of this in the application notes of our blog.

- Impedance: When connecting a signal source, it must be considered the impact that the measuring device will have on the signal. The main consideration is whether the currents going into or out of the analog input will cause noticeable voltage errors due to the impedance of the source. Note that in case of selecting ±12V range for the openDAQ [M], and in all the cases for model [S], the input impedance is about 100kΩ, so only low output impedance signal sources can be connected.
- Speed: Depending on the type of signal and on what post-processing must be done with the readings, different sampling rates will be needed. For instance, it is very different to measure an almost continuous signal than trying to extract peak, average, RMS and frequency from an input waveform. In the case of using multiple channels, changing time must also be considered. OpenDAQ has some limitations with high sampling rates, where only Burst mode can be used. Care should be taken because a too small sampling rate would cause aliasing problems, but higher sampling rates will imply also more noise in the signal, so an intermediate solution would be the optimal choice.

Low level commands

The following are low level functions use to configure and read the analog inputs. You will find details about how to use them and their parameters at the end of the chapter:

- AIN: Read ADC (with actual settings)
- AIN_CFG: Read ADC after configuring analog settings: positive channel, negative channel, gain
- R_CALIB: Read device calibration

Analog Output (DAC)

OpenDAQ has a single analog output, based on a digital to analog converter (DAC). It can be set to any voltage between -4.096V and +4.096V, working with 14 bits of resolution (from 0V to +4.096V, 12bits of resolution, in case of openDAQ [S]).

The analog output commands are sent also as raw binary values. For all device models a 16-bit range is used, so that the code 32768 corresponds to the maximum output voltage. For a desired output voltage, the binary value can be approximated as:

RAW = (Volts/Vref)*32768

Where RAW is the raw binary value that must be sent to the device to setup an output voltage, and Vref is the maximum output value (4.096V in all the cases).

Like in the case of the analog inputs, this formula is only theoretical, and a calibration line must be applied to calculate the adequate raw binary values that product the desired voltage outputs. See software documentation in the repositories for more information.

The analog output can withstand a continuous short-circuit to ground, even when set at maximum output. Voltage should not be applied to analog output, as it is also a voltage source.

Like the analog inputs, analog output can be used in Command-Response mode and also in Stream modes. The firmware use structures called DataChannels to preload an array of points to be used by the DAC.

In this way, and by using a Stream Experiment with analog output, typical waveform signals can be produced from the analog output: sine, triangle, ramp waves, etc.

Low level commands

You will find more details at the end of the chapter:

- SET_DAC: Configure output voltage (-4096 to +4096mV)

Digital I/O

The openDAQ has 6 DIO (digital Inputs/Outputs). We have 4 DIO lines on the right side screw terminal block (D1-D4), and the two others on the left terminal block (D5-D6).

D5 is a multipurpose terminal that is connected with internal microprocessor's Timer/Counter 2. Apart from being used as a DIO, this terminal can be configured as PWM output, Counter input or Capture input.

All the digital I/O lines include an internal series resistor and a protective diode that provides overvoltage/short-circuit protection. The series resistors (about 100Ω) also limit the ability of these lines to sink or source current.

The DIOs have 3 possible states: input, output-high, or output-low. Each line of I/O can be configured individually. When configured as an input, the line has a 50kΩ pull-up resistor to 5.0 volts. When configured as output-high, the line is connected to the internal 5.0 volt supply (through a series resistor). When configured as output-low, a bit is connected to GND (through a series resistor). All digital I/O are configured to be inputs at power up.

Low level commands

We have two couples of commands to control the digital I/O lines. The first two ones control each line individually, one to set or read the line direction (input or output), and the other to read or set the line value (high or low). The other two commands control the six lines at a time, one function to read or set the lines direction, and the other command to read or set the lines values.

- PIO: Write/read PIO output: 1 or 0
- PIO_DIR: Configure PIO direction: 1 input, 0 output
- PORT: Write/read the port including all PIOS
- PORT_DIR: Configure/read all PIOs direction

You will find more details at the end of the chapter.

Capture Input

As described in previous chapters, Capture input can be used to determine the amount of time that an input digital signal spends in high state, low state, or the length of its full period.

Capture input uses ATMEGA 644P internal Timer/Counter 1, and it is connected to D5.

User must initialize this peripheral with an approximate indication of the times that are supposed to be measured, as firmware functions have to preload internal timer with adequate prescaler and divider options, in order to obtain an accurate reading.

Results returned by this peripheral are indicated directly in microseconds.

Low level commands

- CAPTURE_INIT: Starts capture mode around a given period
- CAPTURE_STOP: Stops capture mode
- GET_CAPTURE: Gets actual period length: 0 (low cycle), 1(high cycle), 2(full period)

PWM Output

The PWM uses another of the features of the Timer 1. The user must pass a value for the desired frequency, in microseconds, and the device calculates adequate internal settings in order to output that frequency with high accuracy. Duty value must be passed in a value from 0 to 1023, being the first the value for an always low signal and the last the value for an always high signal.

PWM output is connected to port D5 of openDAQ.

Low level commands

- PWM_INIT: Init PWM with a given period (in us) and duty (/1024)
- PWM_STOP: Disable PWM
- PWM_DUTY: Configure PWM duty when it is already running

Counter Input

The counter input is also based on Timer 1, and its functionality consists on counting number of edges coming through the port (D5). This can be useful to measure the frequency of very fast signal or to read any kind of sensors.

User can select which kind of digital edges will the peripheral detect (high or low), and he can also read and reset the counter back to 0 whenever it is necessary.

The values from the counter are stored in a 32-bit register.

Low level commands

- COUNTER_INIT: Initializes the edge counter (0 high-to-low edges, 1 low-to-high edges)
- GET_COUNTER: Gets counter value (resets accumulator if passed a value)

Encoder Input

The encoder input is based on external interrupts on pin D6. Its functionality consists on counting number of edges coming through the digital input D6 while keeping track of the direction of the movement, by reading D5 on each interrupt.

User can select the maximum resolution of the encoder.

Low level commands

- ENCODER_INIT: Initializes the encoder readings. It configures external interrupts on D6 and resets the pulse counter to 0. Selects encoder resolution.
- ENCODER_STOP: Disables external interrupts on D6.

- GET_ENCODER: Gets encoder value, adjusting the value by the maximum resolution.

Bit-bang SPI Output

(Since firmware version 1.1.3)

The Serial Peripheral Interface (SPI) is a very popular communications bus, used widely in electronics to control slave devices. This utility allows openDAQ to communicate with other low level devices, like external port expanders, PGAs, switches or other peripherals.

SPI is a synchronous serial data link that operates in full duplex mode, using a master/slave scheme, where the master device always initiates the data frame. Multiple slave devices are allowed with separated select lines.

The SPI bus specifies four logic signals:

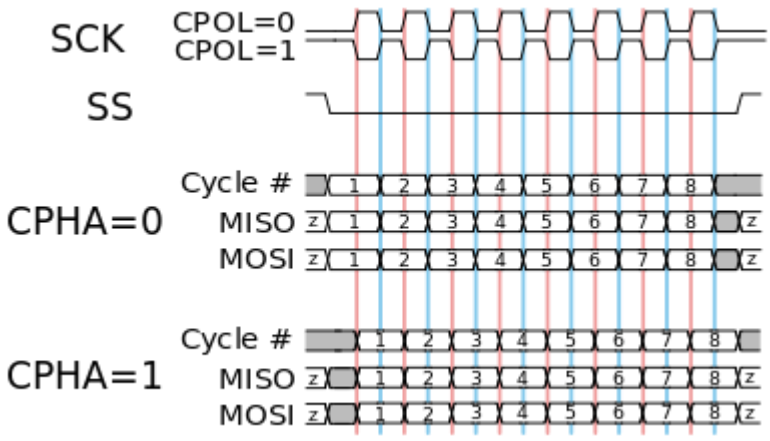
- SCLK: serial clock (output from master).
- MOSI: master output, slave input (output from master).
- MISO: master input, slave output (output from slave).
- SS: slave select (active low, output from master).

To begin a communication, the bus master first configures the clock, and then transmits the logic 0 for the desired chip over the chip select line (SS). During each SPI clock cycle, a full duplex data transmission occurs:

- The master sends a bit on the MOSI line, and the slave reads it from that same line
- The slave sends a bit on the MISO line, and the master reads it from that same line

Transmissions may involve any number of clock cycles.

A relevant issue concerning SPI transmissions, is how the SCLK behaves, and when the MISO and MOSI lines should be read. By convention, these options are named CPOL (clock polarity) and CPHA (clock phase). At CPOL=0 the base value of the clock, when inactive, is zero. CPHA=0 means sample on the leading (first) clock edge, while CPHA=1 means sample on the trailing (second) clock edge, regardless of whether that clock edge is rising or falling. Taking this into consideration, we can define up to four SPI modes, by combining the two possible values of each option. We can see more clearly in this illustration:



OpenDAQ uses a so-called bit-bang SPI mode, as the bus signals are generated entirely by software (no specific hardware is used).

Specific commands are available to configure the functions of the pins (which DIO number will be used for each SPI line) and the SPI mode (CPOL and CPHA). The SS lines must be controlled separately, using any of the DIO terminals not configured as SPI line (PIO command must be used).

Low level commands

- SPISW_CONFIG: Clock mode selection. CPHA and CPOL options.
- SPISW_SETUP: Selects which PIO terminals will be used for the SPI transfer (MISO, MOSI and SCLK). PIO directions are configured.
- SPISW_TRANSFER: Performs a bidirectional SPI communication for a given number of bytes (read and write).

Experiments Configuration

As it was explained in firmware chapter, the openDAQ has two main modes of operation: command-response mode, and stream mode. This last permits to set up the device to perform automatic actions in specific moments. We called it Experiments.

We have three kinds of experiments:

- "STREAM" experiment (internal hardware timed, intervals >1ms)
- "EXTERNAL" experiment (external signal timed)
- "BURST" experiment (internal hardware timed, only one channel, intervals >50µs)

OpenDAQ protocol provides controls to configure and execute that kind of measurements.

Experiment commands

The following commands are used to create an experiment, and to configure the period or the type of external edge that it will use:

- STREAM_CREATE: Create stream experiment
- EXTERNAL_CREATE: Create external experiment
- BURST_CREATE: Create burst experiment

The following two commands control the beginning and the end of the stream mode itself. Up to four experiments can stay active at the same time.

If some of the experiments have a hardware trigger setting, they will not start measuring with the STREAM_START command, but they will wait until the trigger condition is met to start performing actions.

Once the experiment has begun, the device will start sending packets of data whenever they are available. Please check the end of the chapter to see how the STREAM_DATA command works. It is a very special command as it comes directly from the device, with no previous request, and it doesn't also expect any answer. Besides, this kind of packets uses "escape characters". It is a special transmit protocol in order to keep communication well synchronized.

STREAM_STOP command will stop any experiment that stays active. The device will also send this command, without waiting for any response, if all the experiments have finished by themselves. This may happen if they are configured for a limited number of points (not continuous operation).

- STREAM_START: Start a stream measurement (with up to four experiments)
- STREAM_STOP: Stop active measurement
- STREAM_DATA: Device writes a packet with measured data from a experiment

DataChannel Commands

Besides configuring the experiments themselves, the following parts of the protocol permit setting up the internal DataChannels connected with the experiments.

CHANNEL_CFG command allow us to set the type of experiment (analog input/output, digital input/output, counte, capture, etc.), and to set up the ports and the gain that it will use.

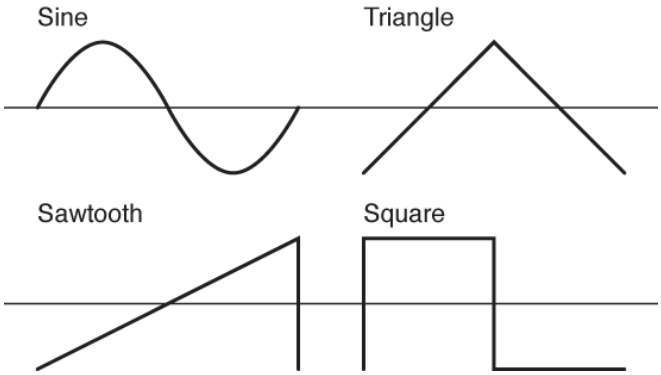
CHANNEL_SETUP is used to configure the experiment total number of points and number of repetitions, while TRIGGER_SETUP sets up the type of trigger used to launch the experiment once the STREAM_START command is executed.

- CHANNEL_SETUP: Configure channel number of points, repeat mode
- CHANNEL_CFG: Configure one of the measurement channels (analog +IN,-IN, GAIN)
- TRIGGER_SETUP: Configure channel trigger mode
- CHANNEL_FLUSH: Resets buffer of data in the channel

Waveform generation

OpenDAQ firmware has a powerful feature that allows the possibility to generate predefined waveforms through DAC output using Stream Mode. The points of a single cycle of a periodic waveform can be charged into the memory of one of the DataChannels and then, when the experiment is carried out, the periodic signal will be displayed on DAC output.

EasyDAQ demos take advantage of this feature and enable users to preload typical periodic waveforms into Channel 4 memory, to be displayed later on. The demo also allows loading the output data points from an external text archive (CSV), and loading the waveform into Burst Mode, which makes possible to display faster signals. The possibilities are wide using this feature. The software demo has the possibility to generate typical periodic signals like sine, square, triangle or sawtooth waveforms:



- Sine: Senoid waveform with selectable amplitude, offset and period. In Stream Mode shorter intervals of minimum 1ms can be used, so the points are calculated taking this into account. For

longer periods, a maximum of 150 points of data will be preloaded in the memory, so the optimal interval between points will be calculated to use all the points.

- Square: Squared waveform with selectable amplitude, offset and period. During time ON, the output of the signal will be offset + amplitude, while during time OFF the output will be offset. The software will calculate the maximum interval between points to allow time ON and time OFF being reached with enough precision.

- Sawtooth: Sawtooth signal with selectable period, amplitude and offset. The interval within points wil be 1ms if the total period is smaller than 140ms. If the period is larger, it will be divided into a maximum of 140 steps, using an interval multiple of 1ms. Potential step between points will be calculated to match the number of total steps in the waveform.

- Triangle: Triangle waveform with selectable period, amplitude, offset and also rise time. Like the other signals, the interval between points will be calculated so that the number of points in a cycle is not greater than 140, and taking into account that the minimum interval is 1ms.

- Import CSV: Imports the points (voltage levels in millivolts) of a text file. The values can be separated by ',' or by line endings. The only selectable option for CSV importing is period. The interval within the points of the file will be the period divided by the total number of points.

Only one DataChannel can be active at a time for Burst Mode, so if this mode is selected for Waveform Generation, no analog input experiment can be configured at the same time. Burst waveforms are selected in the same way as stream waveforms, but the intervals within points of the signal will be calculated to their exact value in microseconds (instead of a multiple of 1 ms), so much faster signals can be generated.

Waveform generation Commands

Waveform generation makes use of standard Stream Mode and Burst Mode functionality, with the difference that before the experiment the buffer of the DataChannel must be populated with the signal points to generate. Interval between points and number of points per period are configured with STREAM_CREATE, BURST_CREATE, CHANNEL_SETUP, CHANNEL_CFG and rest of commands.

- SIGNAL_LOAD: Configure the total number of points and load the data into the buffer of the DataChannel.

CRC

Binary protocol of openDAQ makes use of a 16 bit checksum (CRC), first two bytes, to validate the transferred data packets. The CRC is calculated as the sum of all bytes of the packet, excluding the self CRC bytes. The sum is then complemented (exor'ed by 0xFFFF).

The firmware always calculates and sends the CRC bytes, but the device can be configured to accept PC commands without CRC calculation, in order to allow an easier implementation of the protocol in software tests.

CRC enabling Commands

- ENABLE_CRC: Enable or disable CRC check in the firmware for received commands. If CRC is enabled, bad checksums will be replied with a NACK for every command.

Calibration

AIN and DAC commands are transmitted between the host PC and the device with the raw binary values used in the internal transmission with the peripherals.

These numbers must be operated in order to get the actual value, in millivolts, that is being read through the ADC or to convert a desired output value in millivolts to become the actual voltage for the DAC output. For these calculations to be done, a good approximation is to suppose that the actual values are linear functions of the raw values.

In case of openDAQ-[M] the raw values of the ADC inputs are function of the gain selected, so we will have a different calibration line for each gain setting. On the other hand, we can suppose that all the inputs A1-A8 share the same calibration line, as the signals are multiplexed and then go through the same analog circuitry.

Position	Calibration coefficients
0	DAC
1	AIN1
2	AIN2
3	AIN3
4	AIN4
5	AIN5
6	AIN6
7	AIN7
8	AIN8
9	±12V ADC range
10	±4V ADC range
11	±2V ADC range
12	±0.4V ADC range
13	±0.04V ADC range

In case of openDAQ-[S] the raw values that the ADC returns, are function of the analog input selected, because each resistor bridge will have its own tolerance deviations. Then, we will have a different calibration line for each input setting: one for each analog input configured as SE reading, and another one for each input in SE mode. On the other hand, we can suppose that all the PGA values share the same calibration line. This is because they are applied inside the ADC converter, and the ADS7871 internal PGA values are very well fitted.

Position	Calibration coefficients
0	DAC
1	A1 (SE)
2	A2 (SE)
3	A3 (SE)
4	A4 (SE)
5	A5 (SE)
6	A6 (SE)
7	A7 (SE)
8	A8 (SE)
9	A1-A2 (DE)
10	A2-A1 (DE)
11	A3-A4 (DE)
12	A4-A3 (DE)
13	A5-A6 (DE)
14	A6-A7 (DE)
15	A7-A8 (DE)
16	A8-A7 (DE)

OpenDAQ makes use of the internal microcontroller permanent EEPROM memory to store the coefficients for the linear calibration functions. All values are stored as 16bit integers. Thus, the line coefficients must be converted into integer values prior to be stored in the memory of the device. See “daq.py” documentation to check out how this works.

Calibration Commands

- READ_CALIB: Read device calibration values
- SET_CALIB: Write device calibration values
- C_RESET_CALIB: Restart device calibration values to their default ones

Other Commands

- RESET: System reset and restart
- R_CONFIG: Read device configuration: serial number, firmware version, hardware version
- WAIT_MS: Do nothing until a time has elapsed (milliseconds)
- LED_W: LED configuration: 0 off, 1 green, 2 red, 3 orange

Interacting with openDAQ

Installation on Windows

The first step to start using openDAQ with your computer is having the device drivers installed, so that the computer can recognize it. The openDAQ uses CP2102 USB to UART Bridge to build a Virtual COM Port (VCP) with the computer, in order to facilitate host communication and for reasons of compatibility with the Arduino platform. This means that it is seen like a regular serial port for the host computer.

Some operating systems would recognize the device directly, and you will only need to wait some seconds while the computer updates drivers to start using it. In any other case, you will find necessary drivers in the following link:

http://www.silabs.com/products/mcu/Pages/USBtoUARTBridgeVCPDrivers.aspx

Git repositories

OpenDAQ makes use of git repositories to get track of version control and offer users up to date code and libraries. All source code for Python, LabVIEW, and the firmware of the device are maintained in GitHub, a web-based hosting service which is free for open projects like openDAQ. Note that you will need to register in GitHub (https://github.com/) to upload any source code that you think that would be useful for the project.

Python Libraries and Examples

In our web page users can find some libraries and examples to control openDAQ from Python. Python is a high-level interpreted programming language. It has become very popular during the last few years, especially as a scripting language, although it also can be used to generate standalone executables. Most important, CPython, the reference implementation of Python, is free and open source software and has a community-based development model, as do nearly all of its alternative implementations.

Be aware that there are two versions of Python available right now, Python 2 and Python 3. Although the last is newer and has some improvements, it has not fully compatibility with all operating systems, and many third party tools are not available yet for it. For these reasons, openDAQ libraries and examples are only compatible with Python 2.X.

You can find and download IDE and interpreter here:

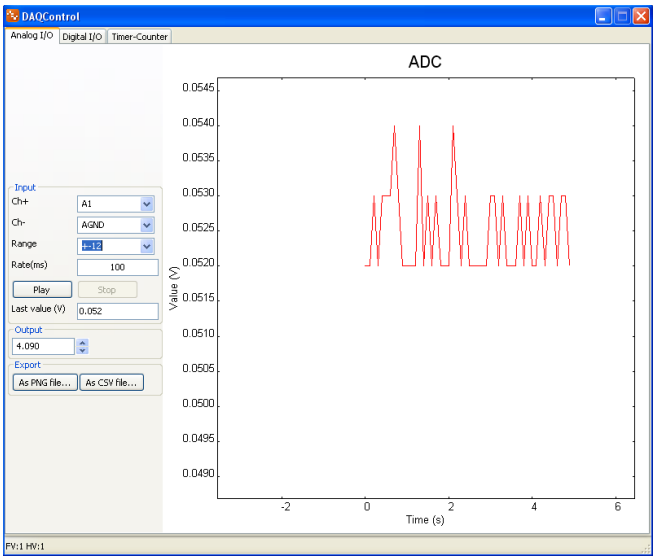
http://python.org/download/

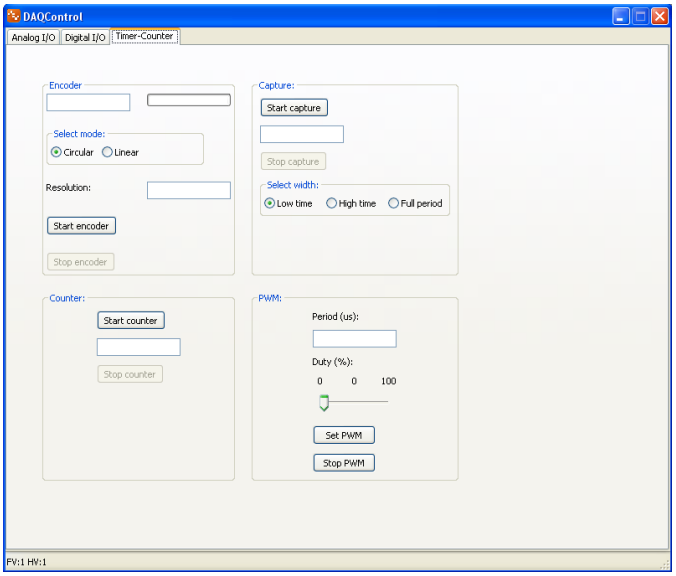
In the Python Git repository available for openDAQ, you will find the following stuff:



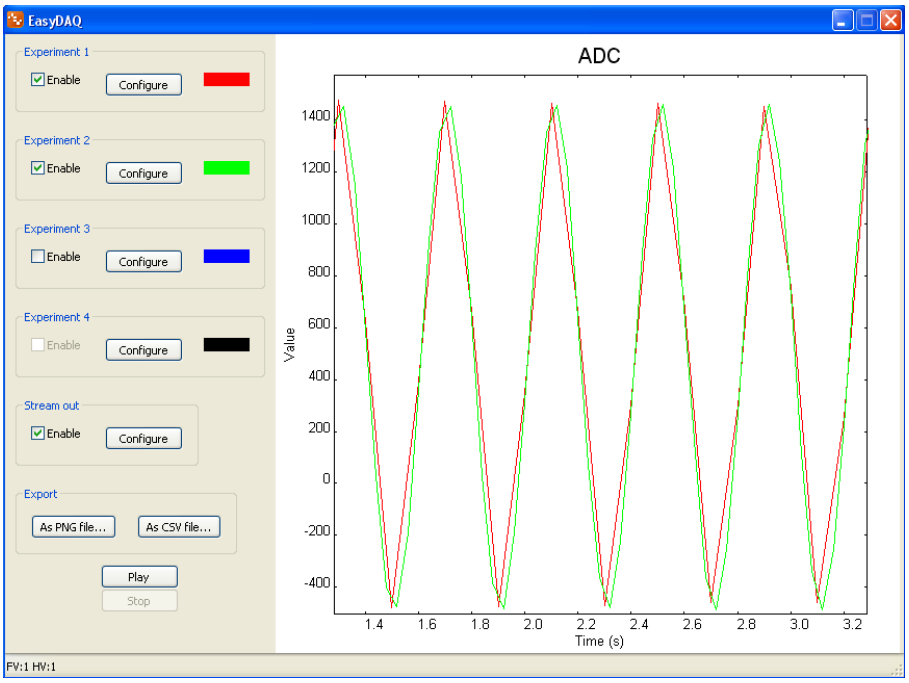
- "Daq.py": This is the low level library for openDAQ, a complete set of functions for communicating the computer with the device. All the other demos and programs make use of this library to communicate with openDAQ. You can also import it from the command prompt to communicate directly with openDAQ. Feel free to use or even modify it for building your own programs.

- DAQcontrol: This is a demo program of complete test panels. It has user controls to access via Comand-Response mode to most of the device functions, including: Analog Inputs, Analog Output, Digital I/Os, and Timer/Counter functions (PWM, Capture and Counter).





- EasyDAQ: This is another demo program, which provides an easy way to configure and perform stream mode experiments. User can configure analog line, gain and scan rate for up to four simultaneous experiments.



- "Calibration.py": This is a utility program, used to obtain the calibration values that are stored into the device's memory.

In order to work with openDAQ and Python, you may need additional modules that you will find in the following links:

http://www.scipy.org/Download (numpy)

http://pypi.python.org/pypi/pyserial (pyserial)

http://www.wxpython.org/download.php (WXPython)

https://github.com/matplotlib/matplotlib/downloads (Matplotlib)

Working environment IDE & compilation

OpenDAQ is a free hardware device, and one of the most interesting things that you could do with this instrument is to modify its internal firmware. It is based on Arduino, the popular open hardware platform.

The first thing that you have to do in order to modify the firmware is to install the Arduino IDE in your computer (http://arduino.cc/en/Main/Software).

A couple of files have to be modified in the Arduino system directories to configure it and run with the openDAQ board:

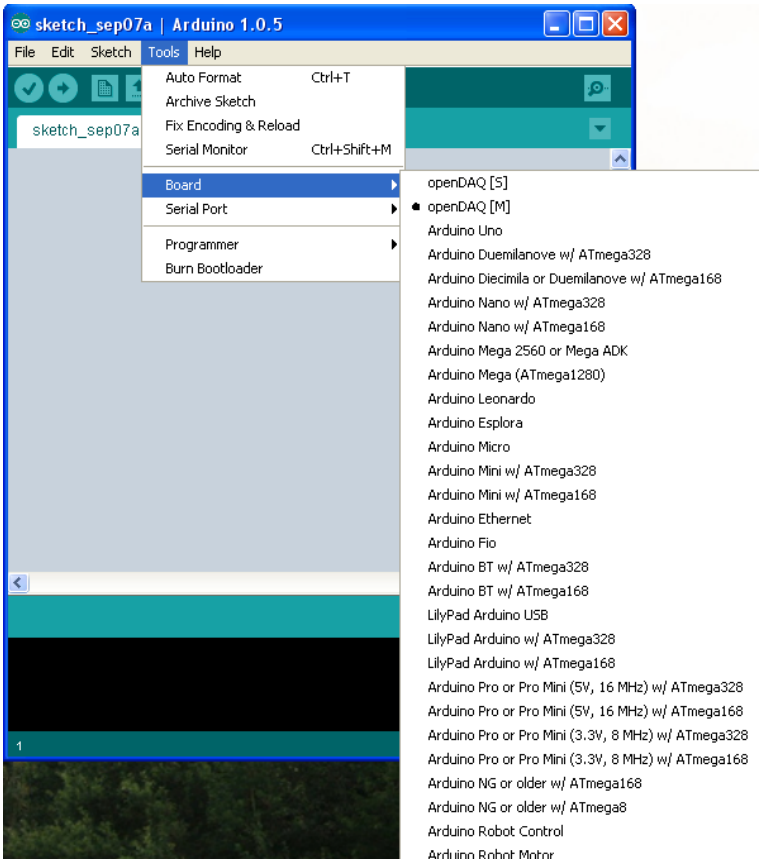
- Add the openDAQ boards definition to the end of the boards.txt file (see "Arduino" folder in Github repositories), into the "core" folder of Arduino's installation route.

- Create new "variant" folders, (openDAQ_s and openDAQ_m), both containing the pin definition file pins_arduino.h that you will also find in the repositories.

- Select the appropriate board (openDAQ [S] or [M]) in the Arduino IDE boards definition.

After following those steps, you will be ready to open the main archives of the openDAQ's firmware, modify them, and upload it into the device.

The Arduino IDE modified for openDAQ can be seen in the following figure:



- In the top part there is a menu bar which allows configuration of general parameters such as the selected serial port.
- Bellow from the menu, there is a button bar which allows compilation, opening, saving or loading the selected code on the board.
- The main part of the program window is occupied by the code editor. Source code to be loaded in the device will be seen in this part.
- On the bottom of the window there is a black status bar that shows us the possible compilation and load errors, as well as the success messages if the process is carried out satisfactorily.

Some examples of alternative firmware codes for openDAQ can be loaded from the File menu. In order to be able to load a new firmware in the device, you have to take care of selecting openDAQ board in the 'Tools/Board' tab. In the 'Tools/Serial Port' tab, the USB to which openDAQ has been connected to the computer must be selected.

Like every Arduino program, the structure of the codes is divided into 2 basic parts: a part named setup and a part called loop.

The setup is the first part of the code, which only runs once when the code is initialized. In this part it is recommendable to include initialization of everything that is going to be used, as well as the part of the

code which is only important when openDAQ is started. The part named loop runs continuously, forming an infinite loop.

You will find a lot of information about this IDE and the most important functions in the Arduino website:

http://arduino.cc/

Electrical characteristics – openDAQ [M]

	Condition	Min	Typical	Max	Units
General					
USB Cable Length				0.8	meters
Supply Voltage		4.75	5	5.25	V
Supply Current			40		mA
Operating Temperature		-40		85	°C
Clock Error	~ 25 °C			±30	ppm
Typ. Command Execution Time		2			ms
+5V Output					
Typical Voltage	Self-Powered	4.75	5	5.25	V
Maximum Current			400		mA
Analog Inputs					
Max AIN Voltage to GND	Valid readings	-12.288		12.288	V
Max AIN Voltage to GND	No Damage	-20		20	V
Input Bias Current			35	65	nA
Input Impedance (Differential)			1.3||10		MΩ||pF
Input Impedance (Common mode)			2.6||15		MΩ||pF
Integral Linearity Error	Range = ±4V			±0.05	% FSR
Absolute Accuracy	Range = ±4V			±0.1	% FSR
Temperature Drift			±10	±35	ppm/°C
Analog Outputs (DAC)					
Nominal Output Range	No Load	-4.096		4.096	V
Resolution			14		bits
Absolute Accuracy	5% to 95% FS		±0.25		% FSR
Differential Linearity Error			±0.1	±0.5	bits
Souce Impedance			100		Ω
Short Circuit Current			±40		mA
Conversion time			4		µs
Digital I/O, Timers, Counters					
Low Level Input Voltage		-0.5		1	V
High Level Input Voltage		3.5		5.5	V
Maximum Input Voltage Range		-0.5		5.5	V
Output low voltage		0		0.9	V
Output high voltage		4.5		5.5	V
Output impedance			100		Ω

Electrical characteristics – openDAQ [S]

	Condition	Min	Typical	Max	Units
General					
USB Cable Length				0.8	meters
Supply Voltage		4.75	5	5.25	V
Supply Current			30		mA
Operating Temperature		-40		85	°C
Clock Error	~ 25 °C			±30	ppm
Typ. Command Execution Time		2			ms
+5V Output					
Typical Voltage	Self-Powered	4.75	5	5.25	V
Maximum Current			400		mA
Analog Inputs					
Max AIN Voltage to GND	Valid readings	-12.288		12.288	V
Max AIN Voltage to GND	No Damage	-20		20	V
Input Impedance (Differential)			83		KΩ
Input Impedance (Common mode)			166		KΩ
Integral Linearity Error		-4	±2	+4	LSB
Differential Linearity Error		-2	±0.5	+4	LSB
Uncalibrated offset error		-24	±1	+24	LSB
Absolute Accuracy	PGA x1			±5	mV
Temperature Drift			±10	±35	ppm/°C
Analog Outputs (DAC)					
Nominal Output Range	No Load	0		4.096	V
Resolution			12		bits
Absolute Accuracy	5% to 95% FS		±3		mV
Integral Linearity Error				±1	LSB
Uncalibrated offset error			±0.5	±5	mV
Souce Impedance			100		Ω
Short Circuit Current			±27	±50	mA
Settling time			4.8		µs
Digital I/O, Timers, Counters					
Low Level Input Voltage		-0.5		1	V
High Level Input Voltage		3.5		5.5	V
Maximum Input Voltage Range		-0.5		5.5	V
Output low voltage		0		0.9	V
Output high voltage		4.5		5.5	V
Output impedance			100		Ω

General and safety info

- OpenDAQ comes mounted in a protective enclosure, to protect it from environmental conditions such as light, dust, humidity or sudden changes in temperature. The naked board should not be installed in any application without this protective enclosure, because the electronic components would be left exposed to the open-air and could become damaged.
- OpenDAQ contains electronic components that are highly sensitive and can be accessed from outside. Handle the device with great care and avoid hitting or scratching any of the surfaces.
- If there is an electrical failure, disconnect the device immediately from the computer or from any other power supply that is being used.
- Keep the device in a dry place and away from any liquids that might splash.
- Never submerge the device into any liquid.
- Do not let the electronic parts come into contact with any steel elements, to avoid injuries and burns.
- Keep the device within the range of temperatures stated in the specifications section.
- Place the device in a location that can only be accessed by maintenance operatives (restricted area).
- Keep children away from the device.

Accessories

There are some available accessories for openDAQ in our web store:

- Test cables
- Test hooks
- Breadboard
- Jumper wires

However, openDAQ is an open system, so feel free to build yourself or buy from third-parties all kinds of accessories, like sensors, connectors, etc.

Warranty

OpenDAQ is covered by a 1 year limited warranty, covering this product and its parts against defects in material or workmanship.

The device can be damaged by bad use of the device, such as connecting VAC to any of the screw terminals. The warranty does not cover damage obviously caused by the customer.

If you have a problem, contact *support@open-daq.com* for return authorization. In the case of warranty repairs, the customer is responsible for shipping back the device to INGEN10 Ingenieria, and INGEN10 Ingenieria will be in charge of the return shipping.

Limitation of liability

Although openDAQ have various electrical protection mechanisms, it is possible, in the case of improper or unreasonable use, to damage the openDAQ and even the PC to which it is connected. Open-Instruments SL will not be liable for such damage.

Except as specified herein, INGEN10 Ingenieria SL makes no warranties, express or implied, including but not limited to any implied warranty or merchantability or fitness for a particular purpose. INGEN10

Ingenieria SL shall not be liable for any special, indirect, incidental or consequential damages or losses, including loss of data, arising from any cause or theory.

INGEN10 Ingenieria SL assumes no liability for applications assistance or customer product design. Customers are responsible for their applications using openDAQ products. To minimize the risks associated with customer applications, customers should provide adequate design and operating safeguards.

Disposal and Recycling

- When openDAQ reaches the end of its useful life, it must be taken to an electronic equipment recycling point.
- The equipment must be disposed of in a selective waste collection system, and not that for urban solid residue. Please manage its disposal properly.

Notes about this manual

- Edited and revised 15/02/2017 by JRB

Certifications

Manufacturer Name: INGEN10 Ingenieria SL

Manufacturer Address: Parque Tecnologico de Asturias. 33428 Llanera, Asturias (SPAIN)

Declares that the product:

- Product Name: openDAQ
- Description of the product: General purpose instrument for software controlled analog and digital acquisition.

Has been built and has undergone final type testing according to the standards:

- Emission: EN/IEC 61326-1: 2006, EN/IEC 61000-6-3:2007, CISPR 11: 2008
- Immunity: EN/IEC 61326-1: 2006, EN/IEC 61000-6-1: 2007, EN/IEC 61000-4-2: 2001, EN/IEC 61000-4-3: 2006

This instrument meets the requirements of the CE mark as contained in the EU directives 2006/95/EC (LVD), 2004/108/EC (EMC).

It fulfills the following specifications:

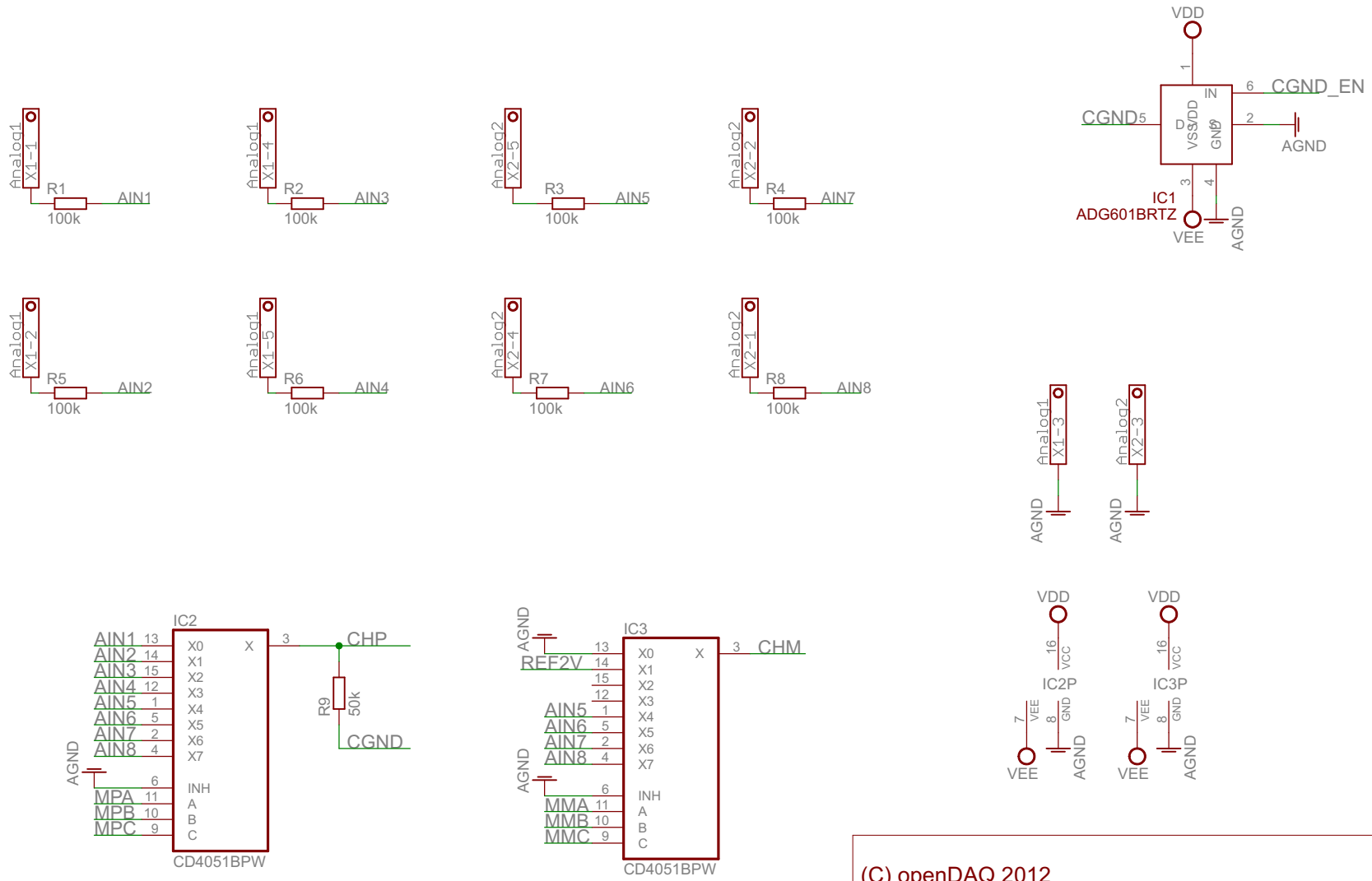
- EN 61326-1: Electrical equipment for measurement, control and laboratory use – EMC requirements

And, therefore, it is marked with CE.

RoHS:

OpenDAQ is RoHS compliant according to the requirements of Directive 2011/65/UE.

DECLARATION OF CONFORMITY



(C) openDAQ 2012

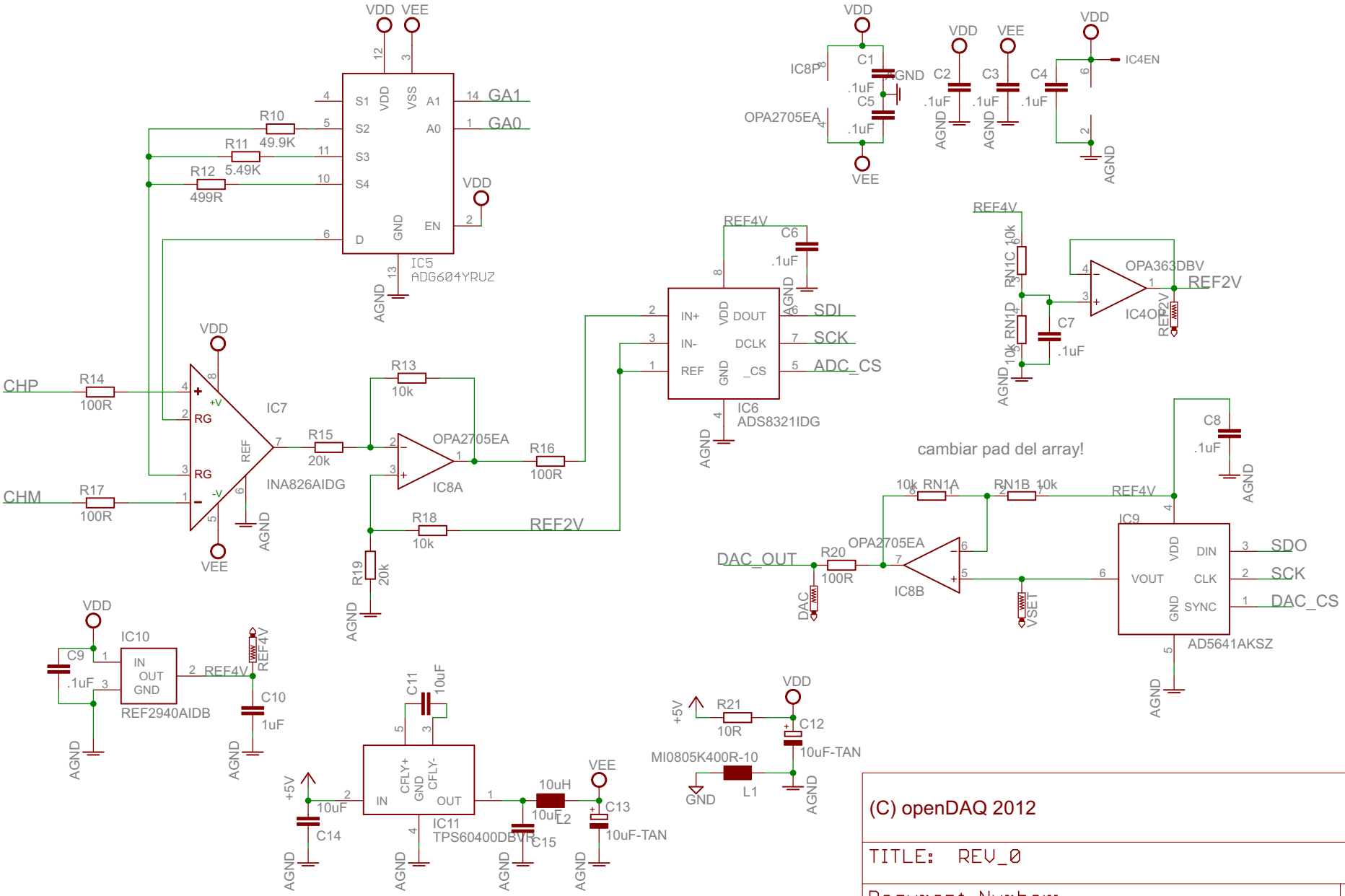
TITLE: REV_0

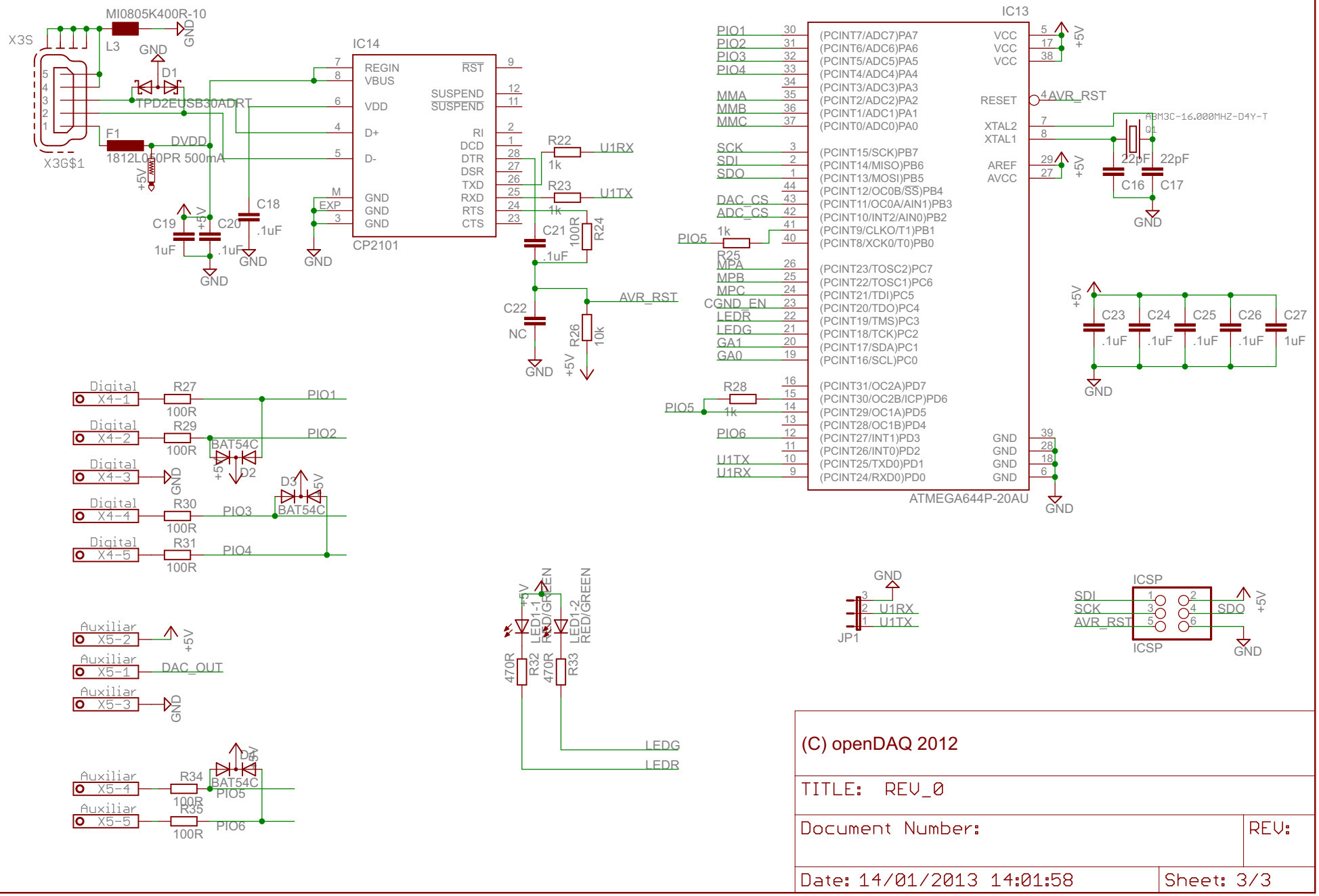
Document Number:

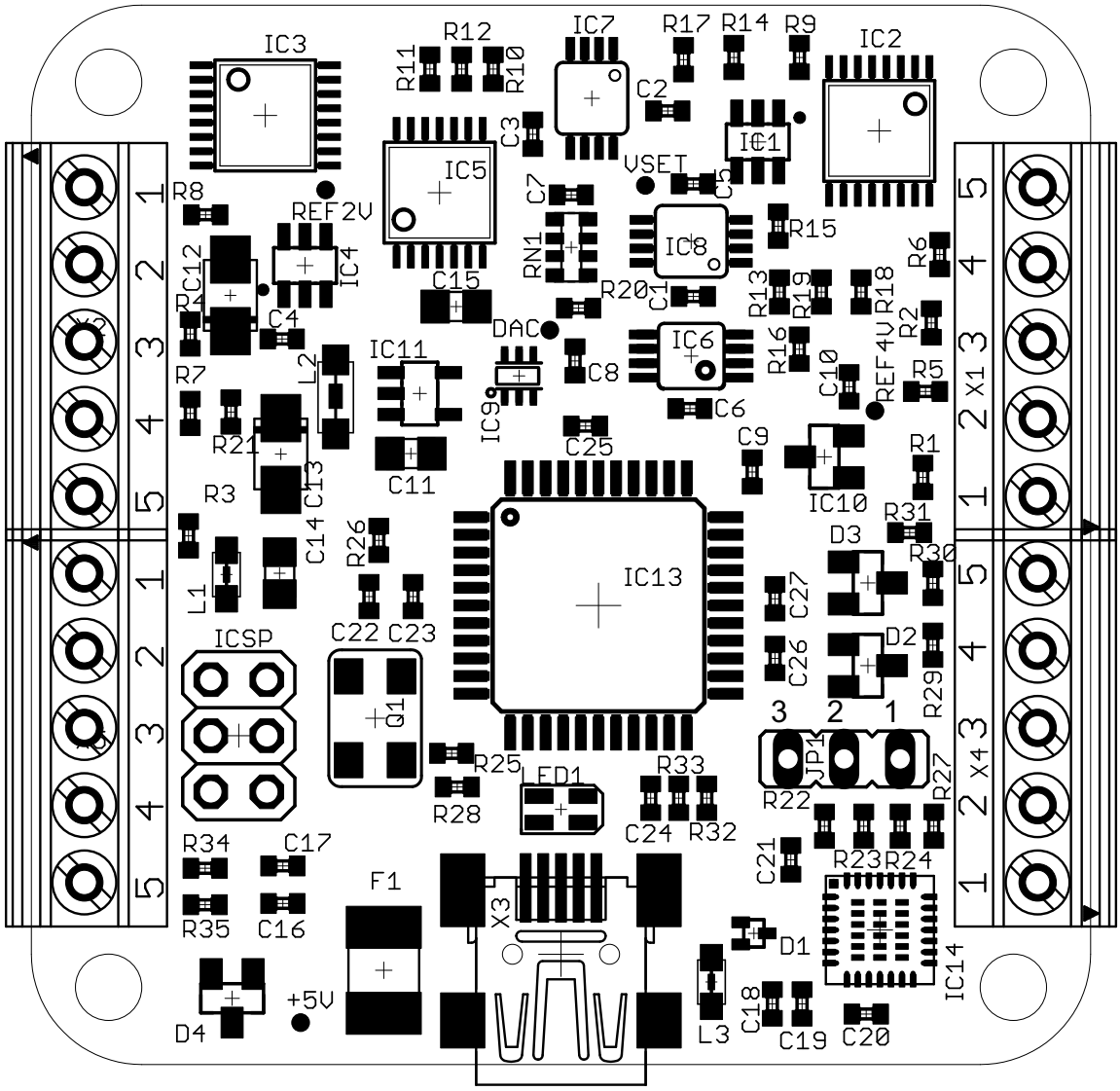
REV:

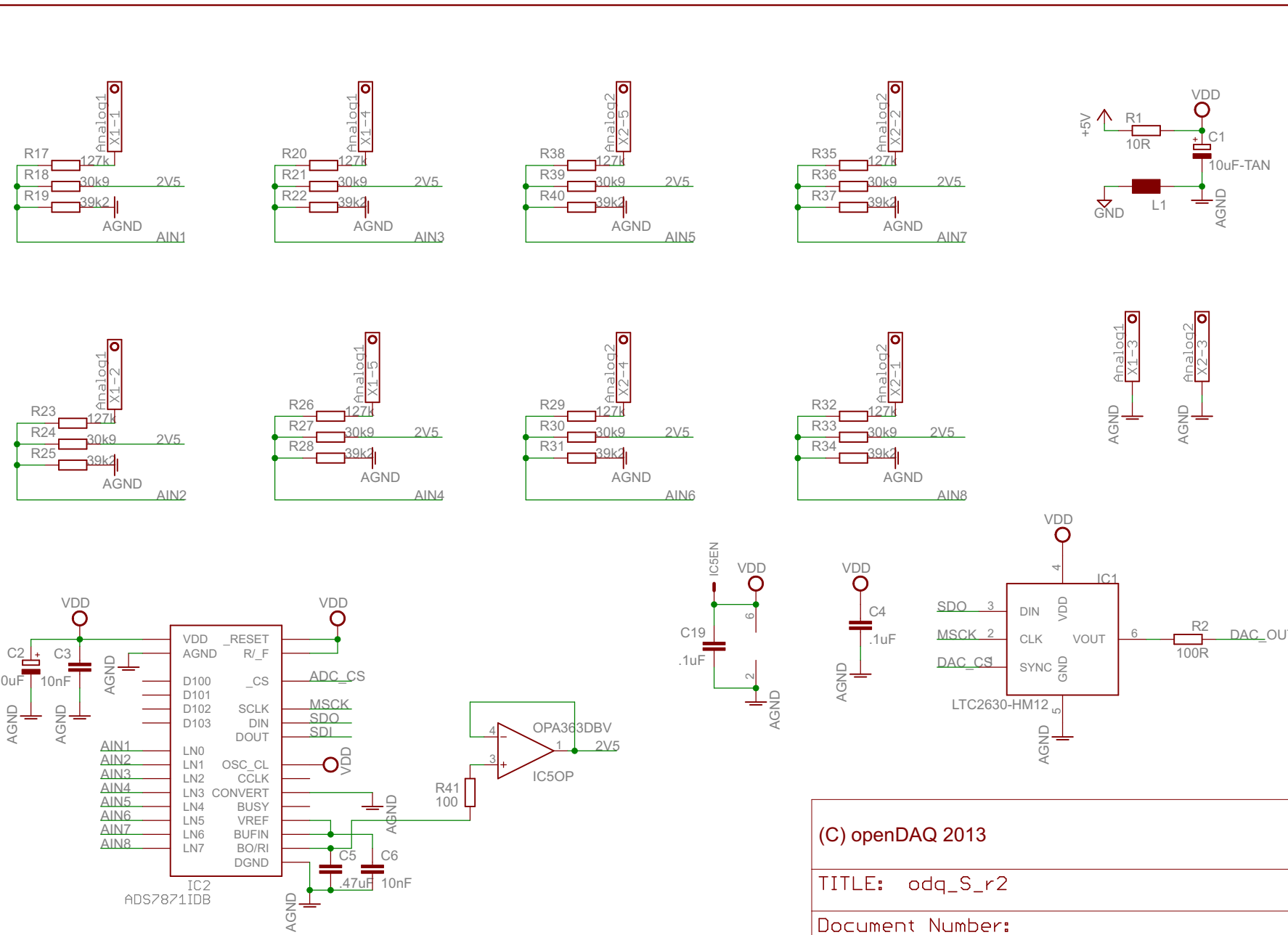
Date: 14/01/2013 14:01:58

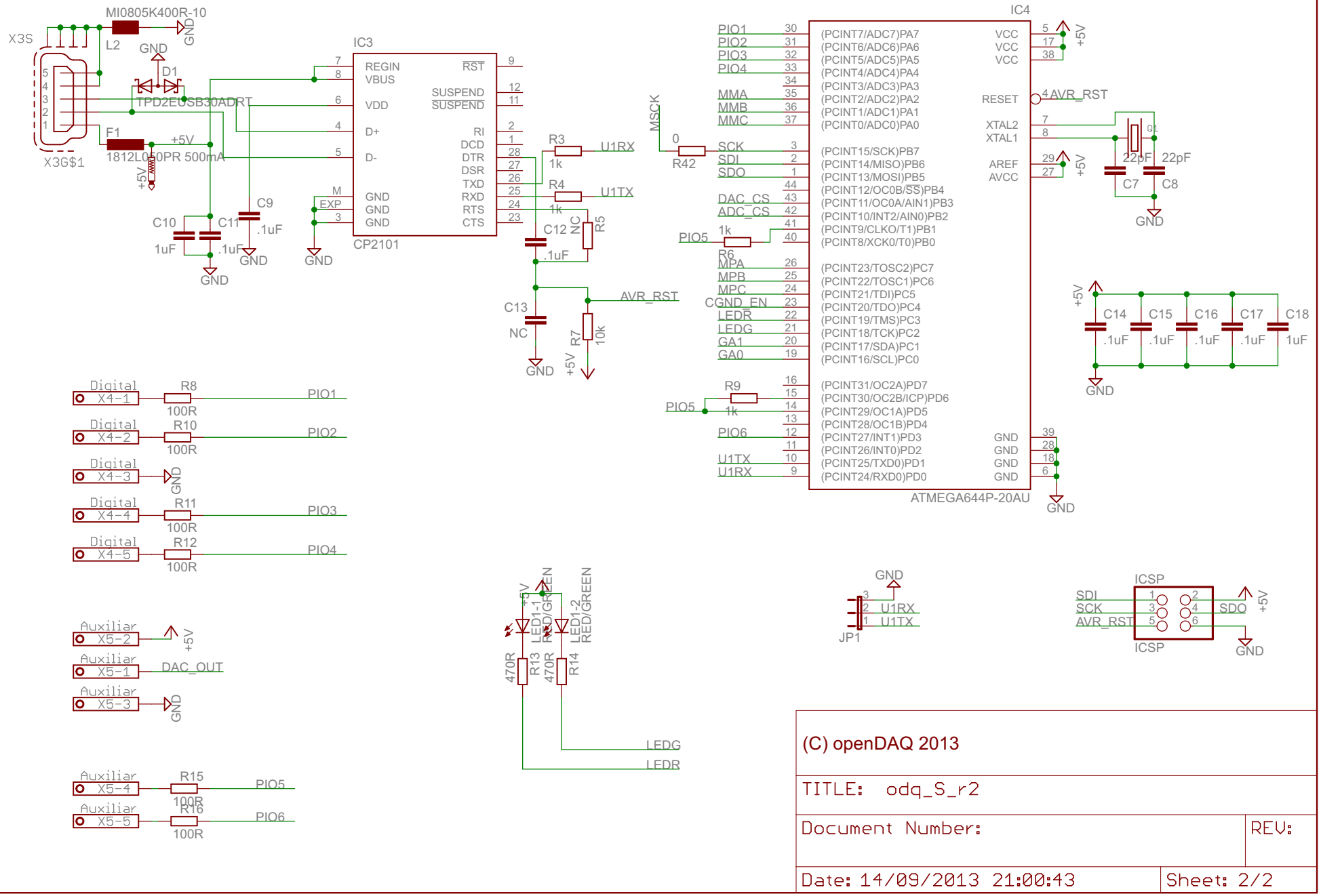
Sheet: 1/3

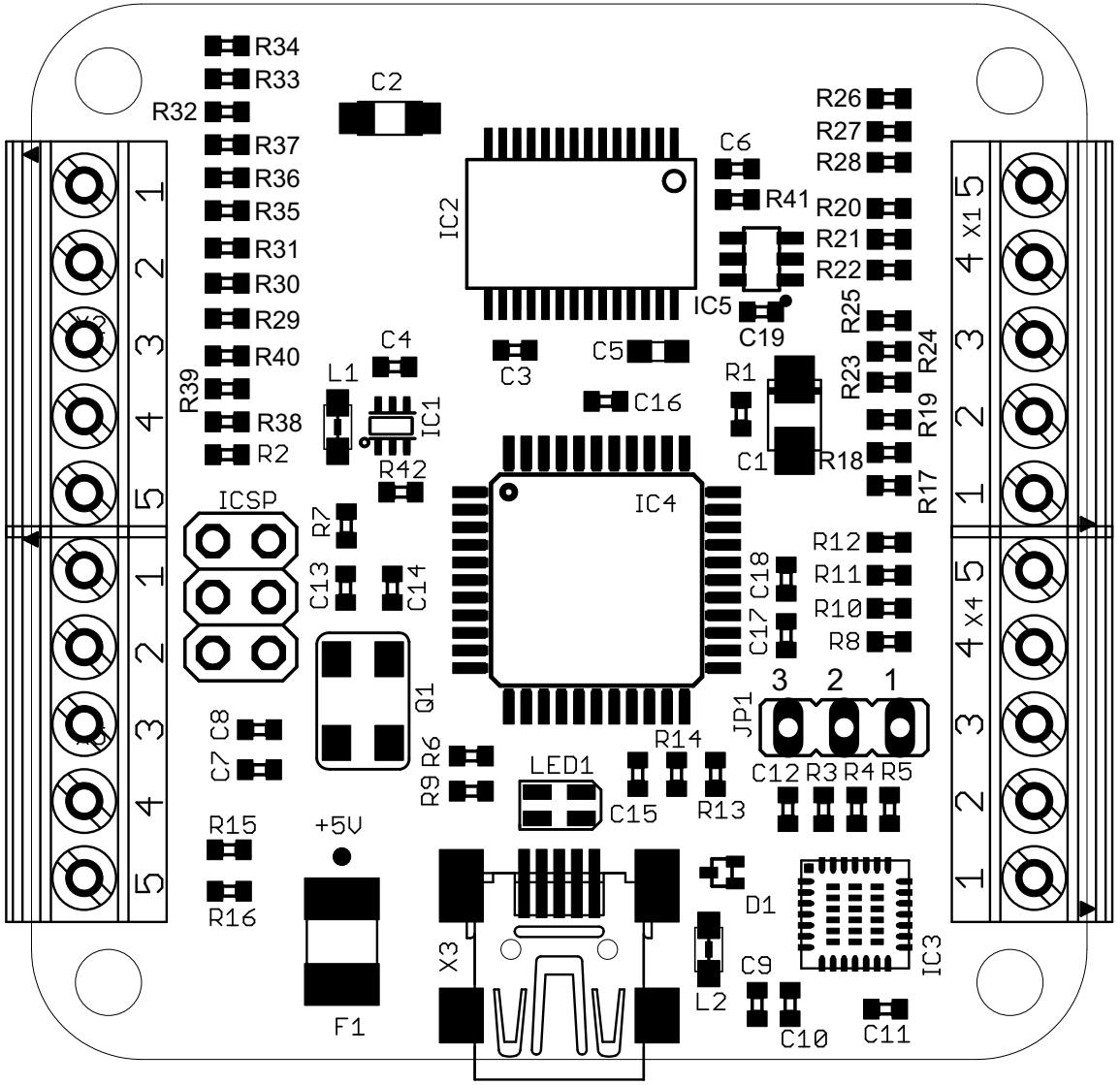

VDD
VEE
IC5
ADG604YRUZ
R10
49.9K
R11
5.49K
R12
499R
GA1
GA0
AGND
IC8B
OPA2705EA
C1
C2
C3
C4
C5
.1uF
IC4EN
IC6
ADS8321IDG
REF4V
C6
SDI
SCK
ADC_CS
IC4OP
OPA363DBVA
REF2V
RN1C 10k
RN1D 10k
C7
CHP
CHM
R14
R17
100R
IC7
INA826AIDG
R13
10k
R15
20k
IC8A
R16
R18
R19
cambiar pad del array!
RN1A 10k
RN1B 10k
C8
IC9
AD5641AKSZ
DAC_OUT
R20
DAC
VSET
SDO
SCK
DAC_CS
IC10
REF2940AIDB
C9
C10
1uF
C11
10uF
IC11
TPS60400DBVR
C14
C15
10uH
L2
C13
10uF-TAN
+5V
R21
10R
C12
MI0805K400R-10
L1
GND
LOGO_OPENDAQ
(C) openDAQ 2012
TITLE: REV_0
Document Number:
REV:
Date: 14/01/2013 14:01:58
Sheet: 2/3


MI0805K400R-10
X3S
L3
GND
D1
TPD2EUSB30ADRT
F1
1812L050PR 500mA
DVDD
X3G$1
+5V
C18
.1uF
C19
1uF
C20
.1uF
IC14
REGIN
VBUS
VDD
D+
D-
GND
GND
GND
RST
SUSPEND
SUSPEND
RI
DCD
DTR
DSR
TXD
RXD
RTS
CTS
CP2101
R22
1k
U1RX
R23
1k
U1TX
C21
.1uF
100R
R24
C22
NC
AVR_RST
R26
10k
IC13
PIO1
PIO2
PIO3
PIO4
MMA
MMB
MMC
SCK
SDI
SDO
DAC_CS
ADC_CS
PIO5
R25
1k
MPA
MPB
MPC
CGND_EN
LEDR
LEDG
GA1
GA0
R28
1k
PIO6
U1TX
U1RX
(PCINT7/ADC7)PA7
(PCINT6/ADC6)PA6
(PCINT5/ADC5)PA5
(PCINT4/ADC4)PA4
(PCINT3/ADC3)PA3
(PCINT2/ADC2)PA2
(PCINT1/ADC1)PA1
(PCINT0/ADC0)PA0
(PCINT15/SCK)PB7
(PCINT14/MISO)PB6
(PCINT13/MOSI)PB5
(PCINT12/OC0B/SS)PB4
(PCINT11/OC0A/AIN1)PB3
(PCINT10/INT2/AIN0)PB2
(PCINT9/CLKO/T1)PB1
(PCINT8/XCK0/T0)PB0
(PCINT23/TOSC2)PC7
(PCINT22/TOSC1)PC6
(PCINT21/TDI)PC5
(PCINT20/TDO)PC4
(PCINT19/TMS)PC3
(PCINT18/TCK)PC2
(PCINT17/SDA)PC1
(PCINT16/SCL)PC0
(PCINT31/OC2A)PD7
(PCINT30/OC2B/ICP)PD6
(PCINT29/OC1A)PD5
(PCINT28/OC1B)PD4
(PCINT27/INT1)PD3
(PCINT26/INT0)PD2
(PCINT25/TXD0)PD1
(PCINT24/RXD0)PD0
VCC
VCC
VCC
RESET
XTAL2
XTAL1
AREF
AVCC
GND
GND
GND
GND
ATMEGA644P-20AU
4AVR_RST
ABM3C-16.000MHZ-D4Y-T
22pF
22pF
C16
C17
C23
C24
C25
C26
C27
.1uF
.1uF
.1uF
.1uF
1uF
Digital
X4-1
Digital
X4-2
Digital
X4-3
Digital
X4-4
Digital
X4-5
R27
100R
R29
100R
R30
100R
R31
100R
BAT54C
D2
D3
BAT54C
PIO1
PIO2
PIO3
PIO4
Auxiliar
X5-2
Auxiliar
X5-1
DAC_OUT
Auxiliar
X5-3
Auxiliar
X5-4
Auxiliar
X5-5
R34
100R
R35
100R
BAT54C
PIO5
PIO6
LED1-1
RED/GREEN
LED1-2
RED/GREEN
470R
R32
470R
R33
LEDG
LEDR
JP1
ICSP
SDI
SCK
AVR_RST
SDO
ICSP
(C) openDAQ 2012
TITLE: REV_0
Document Number:
REV:
Date: 14/01/2013 14:01:58
Sheet: 3/3


IC3
R12
R11
R10
IC7
R17
R14
R9
IC2
C2
C3
IC1
IC5
VSEI
C7
C5
R8
REF2V
R15
IC8
RN1
R6
C12
IC4
C15
R20
C1
R13
R19
R18
R4
C4
DAC
R2
IC11
IC6
R16
C10
REF4V
R5
R7
L2
C8
X1
IC9
C6
R21
C25
C9
R1
C13
C11
IC10
R3
R31
C14
R26
D3
R30
IC13
C27
L1
C22
C23
D2
C26
ICSP
Q1
R29
3
2
1
R25
LED1
R33
JP1
X4
R22
R27
R28
C24
R32
R34
C17
C21
R23
R24
F1
X3
D1
R35
C16
IC14
+5V
C18
L3
D4
C19
C20


Analog1
X1-1
R17
127k
R18
30k9
R19
39k2
2V5
AGND
AIN1
Analog1
X1-4
R20
127k
R21
30k9
R22
39k2
2V5
AGND
AIN3
Analog2
X2-5
R38
127k
R39
30k9
R40
39k2
2V5
AGND
AIN5
Analog2
X2-2
R35
127k
R36
30k9
R37
39k2
2V5
AGND
AIN7
+5V
R1
10R
VDD
C1
10uF-TAN
GND
L1
AGND
Analog1
X1-2
R23
127k
R24
30k9
R25
39k2
2V5
AGND
AIN2
Analog1
X1-5
R26
127k
R27
30k9
R28
39k2
2V5
AGND
AIN4
Analog2
X2-4
R29
127k
R30
30k9
R31
39k2
2V5
AGND
AIN6
Analog2
X2-1
R32
127k
R33
30k9
R34
39k2
2V5
AGND
AIN8
Analog1
X1-3
Analog2
X2-3
AGND
AGND
VDD
C2
10uF
C3
10nF
AGND
AGND
AGND
VDD
VDD
_RESET
AGND
R/_F
D100
D101
D102
D103
_CS
SCLK
DIN
DOUT
ADC_CS
MSCK
SDO
SDI
VDD
LN0
LN1
LN2
LN3
LN4
LN5
LN6
LN7
OSC_CL
CCLK
CONVERT
BUSY
VREF
BUFIN
BO/RI
DGND
AIN1
AIN2
AIN3
AIN4
AIN5
AIN6
AIN7
AIN8
AGND
C5
.47uF
C6
10nF
AGND
IC2
ADS7871IDB
R41
100
OPA363DBV
IC5OP
2V5
IC5EN
VDD
C19
.1uF
AGND
VDD
C4
.1uF
AGND
VDD
IC1
SDO
MSCK
DAC_CS
DIN
CLK
SYNC
VDD
VOUT
GND
LTC2630-HM12
AGND
R2
100R
DAC_OUT
(C) openDAQ 2013
TITLE: odq_S_r2
Document Number:
REV:
Date: 14/09/2013 21:00:43
Sheet: 1/2


MI0805K400R-10
X3S
L2
GND
D1
TPD2EUSB30ADRT
F1
1812L050PR 500mA
X3G$1
+5V
C9
.1uF
C10
1uF
C11
.1uF
IC3
REGIN
VBUS
VDD
D+
D-
GND
EXP
GND
GND
RST
SUSPEND
SUSPEND
RI
DCD
DTR
DSR
TXD
RXD
RTS
CTS
CP2101
R3
1k
U1RX
R4
1k
U1TX
C12
.1uF
NC
R5
C13
NC
R7
10k
AVR_RST
MSCK
0
R42
R6
1k
PIO5
DAC_CS
ADC_CS
MMA
MMB
MMC
IC4
(PCINT7/ADC7)PA7
(PCINT6/ADC6)PA6
(PCINT5/ADC5)PA5
(PCINT4/ADC4)PA4
(PCINT3/ADC3)PA3
(PCINT2/ADC2)PA2
(PCINT1/ADC1)PA1
(PCINT0/ADC0)PA0
(PCINT15/SCK)PB7
(PCINT14/MISO)PB6
(PCINT13/MOSI)PB5
(PCINT12/OC0B/SS)PB4
(PCINT11/OC0A/AIN1)PB3
(PCINT10/INT2/AIN0)PB2
(PCINT9/CLKO/T1)PB1
(PCINT8/XCK0/T0)PB0
(PCINT23/TOSC2)PC7
(PCINT22/TOSC1)PC6
(PCINT21/TDI)PC5
(PCINT20/TDO)PC4
(PCINT19/TMS)PC3
(PCINT18/TCK)PC2
(PCINT17/SDA)PC1
(PCINT16/SCL)PC0
(PCINT31/OC2A)PD7
(PCINT30/OC2B/ICP)PD6
(PCINT29/OC1A)PD5
(PCINT28/OC1B)PD4
(PCINT27/INT1)PD3
(PCINT26/INT0)PD2
(PCINT25/TXD0)PD1
(PCINT24/RXD0)PD0
VCC
VCC
VCC
RESET
XTAL2
XTAL1
AREF
AVCC
GND
GND
GND
GND
ATMEGA644P-20AU
22pF
22pF
C7
C8
C14
C15
C16
C17
C18
.1uF
.1uF
.1uF
.1uF
1uF
R9
1k
PIO6
U1TX
U1RX
Digital
X4-1
R8
100R
PIO1
Digital
X4-2
R10
100R
PIO2
Digital
X4-3
Digital
X4-4
R11
100R
PIO3
Digital
X4-5
R12
100R
PIO4
Auxiliar
X5-2
Auxiliar
X5-1
DAC_OUT
Auxiliar
X5-3
Auxiliar
X5-4
R15
100R
PIO5
Auxiliar
X5-5
R16
100R
PIO6
LED1-1
RED/GREEN
LED1-2
RED/GREEN
R13
470R
R14
470R
LEDG
LEDR
JP1
U1RX
U1TX
ICSP
SDI
SCK
AVR_RST
SDO
ICSP
(C) openDAQ 2013
TITLE: odq_S_r2
Document Number:
REV:
Date: 14/09/2013 21:00:43
Sheet: 2/2


R34
R33
R32
R37
R36
R35
R31
R30
R29
R40
R39
R38
R2
C2
C4
L1
IC1
R42
C3
IC2
C6
R41
IC5
C19
C5
C16
R1
C1
R18
R26
R27
R28
R20
R21
R22
R25
R23
R24
R19
R17
X1
X4
ICSP
R7
C13
C14
IC4
C18
C17
R12
R11
R10
R8
3 2 1
JP1
Q1
C8
C7
R6
R9
R14
LED1
C15
R13
C12
R3
R4
R5
R15
R16
+5V
F1
X3
D1
L2
C9
C10
C11
IC3